

Scratch for Parents: Learn It, Then Teach It

The full teaching manual for the six-session workshop series. Timing, live-demo scripts, the grown-up-learner stumbles to pre-empt, lab facilitation, Teach-It role-plays, and checks for understanding — for all five modules.

5 modules • 17 lessons

6 sessions • ~12 hours

Instructor copy • not for participants

CONTENTS

MOD 1 Getting Started with Scratch

- 1.1 What Is Scratch & Why Kids Love It
- 1.2 A Guided Tour of the Scratch Editor
- 1.3 Your First Project — Make the Cat Move & Talk

MOD 2 Core Coding Concepts

- 2.1 Sequences & Timing
- 2.2 Loops — Making Things Repeat
- 2.3 Conditionals & Sensing — Making Choices

MOD 3 Building Real Projects

- 3.1 An Interactive Animated Story
- 3.2 A Simple Game with Variables & Score
- 3.3 Debugging & Remixing — Fixing and Improving

MOD 4 Leveling Up — Bigger Projects

- 4.1 Clones — One Sprite, Many Copies
- 4.2 My Blocks — Make Your Own Blocks
- 4.3 Capstone Part 1 — Plan It & Build the Core
- 4.4 Capstone Part 2 — Lives, Game Over & Launch Day

MOD 5 Teaching Scratch to Your Child

- 5.1 The Art of Not Taking Over
- 5.2 Sharing Safely & the Scratch Community
- 5.3 ScratchJr — Coding for Ages 5–7

5.4 What Comes After Scratch

Module 1 — Getting Started with Scratch

How to run the first session: settle a room of nervous grown-ups, get everyone into the Scratch editor (account or not), tour it as a game, and end with every parent's cat walking and talking. The goal for Session 1 is confidence, not coverage.

3 lessons

2 hrs teaching

Lessons 1.1-1.3

Level Beginner

Maps to Session S1

HOW TO USE THIS GUIDE

- ◆ Same rhythm every lesson: **Frame** → **Learn It** (live demo) → **Lab** → **Teach-It role-play** → **Check**. Participants learn it themselves first, then practice coaching a "child" played by their partner.
- ◆ **Session 1 is mostly about fear.** Your participants are adults with zero coding experience; many haven't been a "student" in decades. Every minute you spend making them feel safe here buys you an hour of attention later.
- ◆ **Demo on the projector, then hands on.** Never ask a room to follow along while you talk fast. Show it once, slowly; then they do it while you circulate.
- ◆ The workbook mirrors this guide. Point participants to the matching page before each lab so the steps are in front of them.
- ◆ **Teal** = hands-on, **amber** = watch-outs. **SAY:** boxes are lines to read aloud, or for participants to use when coaching.

THE THROUGH-LINE

Module 1 runs from "What is this?" to "I made that." Lesson 1.1 gives participants three words (**sprite**, **stage**, **blocks**) and one mindset ("**...yet**"). Lesson 1.2 gives them a map of the editor. Lesson 1.3 gives them their first working program, a cat that walks and says hello. Each lesson also rehearses one parenting move: the *pitch* (1.1), the *60-second tour* (1.2) and *coaching with questions* (1.3). Every parent should leave with a saved project and a first journal entry.

BEFORE SESSION 1 — ROOM & ACCOUNT LOGISTICS

◆ ONE WEEK BEFORE

- ◆ **Test the network from the actual room.** Open `scratch.mit.edu` → **Create**. The page loading isn't enough: check that the cat *appears* and that

◆ ROOM KIT

- ◆ **USB mice**, at least one per three participants. Trackpad drag-and-drop is the #1 physical barrier for adults.

the sprite library (cat "+" button) shows pictures. School and library filters often allow the site but block its asset servers, so the editor opens with no cat.

- ◆ **If anything is blocked:** ask IT to allow `*.scratch.mit.edu`. If they can't do it in time, install the free **Scratch app** (`scratch.mit.edu/download`, for Windows, macOS, ChromeOS and Android tablets) on the room machines. Also bring a USB stick with the installers.
- ◆ **Send the "before you come" note:** bring a laptop and charger if you have one; bring a mouse; bring reading glasses; **no account needed**, but if you want one, sign up at home and confirm the email first.

- ◆ Seats in **pairs** (role-plays start in Lesson 1.1). Put the projector where everyone can see it without twisting round.
- ◆ Printed workbooks, pens, name tents.
- ◆ Printed **kid cards** for the three role-plays in this guide (cut apart, one set per pair).
- ◆ A sticky note of the Wi-Fi password on every table.

ACCOUNTS — WHAT TO TELL THE ROOM

Accounts are optional in Module 1. Anyone can click **Create** and build without signing in. The only difference is saving: without an account, use **File → Save to your computer** (downloads an `.sb3` file); with one, **File → Save now** saves to the cloud.

Parent-email confirmation. When someone joins, Scratch sends a confirmation email: to the adult's own address, or to a parent/guardian's address for a child's account. Until the link in that email is clicked, the account can save but **can't share or comment**. If someone signs up in class, have them confirm on their phone before they leave. Warn that the email sometimes lands in spam or promotions.

Kid accounts are homework, not class work. Parents create them at home, with their child, using their own email as the guardian address and a username that doesn't reveal the child's real name. Sharing safety is covered properly in Lesson 5.2 (Session 6).

Organizations: a school or library running many kids' accounts can apply for a free Scratch **Teacher Account**, which creates student accounts without individual emails. Mention it if a teacher asks, but don't set one up during the session.

MODULE AT A GLANCE

SEGMENT	TOPIC	TIME	PARTICIPANTS LEAVE ABLE TO...
Warm-up	Arrival, devices, accounts + "why are you here?" icebreaker	15m	Sit at a working machine, know the account options, and name why they came

SEGMENT	TOPIC	TIME	PARTICIPANTS LEAVE ABLE TO...
1.1	What Is Scratch & Why Kids Love It	25m	Say what Scratch is in one sentence; pitch it to their child
1.2	A Guided Tour of the Scratch Editor	33m	Name the four zones, find any block, save a project
Break	—	10m	—
1.3	Your First Project: Make the Cat Move & Talk	27m	Build, run, change and fix a three-block program
Wrap-up	Journal + "try at home this week"	10m	Leave with a saved project, a journal entry, and one thing to do at home
Total		120m	Alternative format: as the first half of Saturday 1 in a 3 × half-day series, then run S2 after lunch.

SESSION S1 RUN-SHEET

CLOCK	MIN	SEGMENT	WHAT YOU DO
0:00–0:10	10	Arrival & check-in	Greet at the door; mouse and workbook on every seat. Everyone opens scratch.mit.edu → Create and checks the cat appears. Go over the account options (see the box above). Housekeeping: restrooms, the break at 0:58, "phones on buzz; step out for family calls whenever you need to."
0:10–0:15	5	Icebreaker: "Why are you here?"	Pairs, 90 seconds each: name, child's age, and <i>one</i> answer from the workbook's journal "why page". Then take 3–4 volunteers for the room. You go first.
0:15–0:32	17	1.1 Learn It	What Scratch is · sprite / stage / blocks · blocks vs typed code · color = category · "yet"
0:32–0:38	6	1.1 Teach-It role-play	"The pitch", with kid card A
0:38–0:40	2	1.1 Check	Show of hands on the 4 self-check items
0:40–0:58	18	1.2 Learn It + lab	The four zones · "Point and Name" game · green flag / stop · the wrong-sprite trap
0:58–1:08	10	Break	Stay at the front for account and home-device questions

CLOCK	MIN	SEGMENT	WHAT YOU DO
1:08–1:16	8	1.2 Saving & accounts	Save to your computer + reload it · Save now · kid-account walkthrough (on screen only) · offline app
1:16–1:21	5	1.2 Teach-It role-play	"The 60-second tour", with kid card B
1:21–1:23	2	1.2 Check	Show of hands
1:23–1:31	8	1.3 Live demo	Build the cat script block by block on the projector
1:31–1:43	12	1.3 Lab	Build it, then "Make it yours" challenges; save
1:43–1:48	5	1.3 Teach-It role-play	"Treasure hunt, not dictation", with kid card C
1:48–1:50	2	1.3 Check	Show of hands
1:50–2:00	10	Wrap-up	Journal (5m) · 2–3 shares · "try at home this week" · confidence number 1–10 · save and confirm the file is where you think it is

120

ICEBREAKER: "WHY ARE YOU HERE?" (5 MIN)

This is the Lesson 1.1 "first journal entry" said out loud. It does two jobs: it tells you who's in the room (child ages, hopes, worries), and it lets nervous adults hear that the person next to them is nervous too. In pairs, each person gets 90 seconds to answer **one** of these:

- ◆ **Why** do you want to learn this? (a curious kid, a school requirement, your own itch)
- ◆ **What** do you hope to make with your child? (a game, a dancing birthday card, a story starring the dog)
- ◆ **What worries** you about it? (not techy enough, a frustrated child, screen time)

Model it first with a real, slightly vulnerable answer of your own. When volunteers share worries, **write them on a flip-chart and keep it up all series**. In Session 6 you'll cross them off together. Don't try to solve screen-time guilt now; say "Great one. We'll come back to that in Module 5."

1.1 What Is Scratch & Why Kids Love It

25 min

◆ OBJECTIVES

- ◆ Explain, in one sentence, what Scratch is
- ◆ Describe how block-based coding differs from typed code
- ◆ List three reasons Scratch is great for young learners
- ◆ Introduce Scratch to their child with confidence

◆ TEACHING SEQUENCE

- 1 The one-sentence definition (3m).** Put it on the slide and have the room read it aloud together: *"Scratch is a free website where kids make their own games, animations and stories by snapping together colorful blocks instead of typing code."* Made by the MIT Media Lab, run by the non-profit Scratch Foundation, available in 70+ languages, no ads, no in-app purchases.
- 2 Three words (3m).** Point at the projector: **sprite** (the cat, a character or object), **stage** (the screen where the action happens), **blocks** (the instructions you snap together). "Learn these three and you can already follow along with your child."
- 3 Blocks vs typing, live (5m).** Script: *"In typed code, one missing colon and nothing works. Watch."* Show the Python-style snippet on the slide. Then build **when**  **clicked → move 10 steps → say Hello! for 2 seconds** in about 20 seconds. *"Blocks only snap together if they make sense, so there's nothing to misspell."* Then try to force a hat block *under* a move block to show that it won't go.
- 4 Color = category (2m).** Blue Motion, purple Looks, pink Sound, gold Events, orange Control, light-blue Sensing, green Operators, dark-orange Variables. "Kids read a program by its colors before they can read the words."
- 5 What kids feel vs what they're learning (2m).** "I made the cat dance!" = sequencing; "it keeps spinning forever" = loops; "when I press space it jumps" = events. *"Your job isn't to lecture on these terms. Celebrate the moment, and now and then name it."*
- 6 Growth mindset: "yet" (2m).** Look back at the worries flip-chart. *"'I'm not a computer person' becomes 'I'm not a computer person yet.' Your kids learn how to handle frustration by watching you hit a bug and shrug."*

◆ LAB – CLICK AROUND (FOLDED INTO THE DEMO)

No build file. During step 3, participants have the editor open and drag any one block into the middle and click it. That's the whole lab: prove to themselves that nothing breaks.

Circulate for: anyone frozen with hands off the mouse ("Go on, drag anything, you can't break it"); trackpad strugglers (give them a mouse *now*, not in Lesson 1.3).

◆ **TEACH-IT ROLE-PLAY — "THE PITCH" (6 MIN)**

Partner A plays the child from the kid card; Partner B makes the pitch. **No screen**, just a conversation. Swap after 2 minutes, then debrief for 1 minute.

KID CARD A — MAYA, 9

Mood: Skeptical. Just got home, a bit tired. **Typical behavior:** "Coding is boring, it's just maths." Loves drawing and making up stories about the cat next door. Warms up if the idea is *hers*.

SAY: *"Want to make your own game where you decide the rules? There's a free thing called Scratch where you snap colorful blocks together and a little cat does whatever you tell it. Want to try making it talk?"*

SAY: *"You like drawing. In Scratch you can draw your own character and make it move."*

SAY: *"I'm learning it too, so I'll probably get stuck. Maybe you can help me."*

Debrief question: "Kids, which line made you curious? Which one sounded like school?" (It's almost always the word **learn**. "**Make** beats **learn** every time.")

◆ **WATCH-OUTS**

- ◆ **"I'm not a computer person."** Answer it head-on: "If you can drag a block, you can help." Never say "this is easy". To someone who is struggling that sounds like "so you must be slow."
- ◆ **Comparing with faster peers.** Some participants already have Scratch-mad kids and will race ahead. Give them the side job of "table helper", which turns speed into something useful for the room.
- ◆ **Screen-time guilt** comes up early. Acknowledge it and park it: making things in Scratch is *creative* screen time, and Module 5 is about exactly this.
- ◆ **The community question.** "Can strangers message my kid?" Scratch has a moderated community for sharing and comments, but Module 1 never touches it. Everything today stays private. Full coverage in Lesson 5.2.
- ◆ **ScratchJr confusion.** Parents of 5–7-year-olds will ask. ScratchJr is a separate picture-based tablet app, covered in Lesson 5.3. For now, younger kids sit beside the parent.

◆ **CHECK — QUIZ**

Answers: the biggest advantage of blocks is **no syntax or typing errors that break the program** (B); a block's color tells you its **category** (B); the best first-session goal is one **"whoa, I made that!" moment** (C); *new*: the three key words are **sprite, stage, blocks**, and the cat is a **sprite** (B).

◆ OBJECTIVES

- ◆ Open the Scratch editor, with or without an account
- ◆ Name the four main areas of the editor and what each does
- ◆ Find any block using the colored category menu
- ◆ Use the green flag and the stop button
- ◆ Save their work, and know how to set up a safe kid account

◆ TEACHING SEQUENCE

- 1 **The map (4m).** Show the four-zone slide, then the real editor. **1 Blocks Palette** (left: pick instructions) → **2 Code Area** (middle: snap them together) → **3 Stage** (upper right: watch it happen) → **4 Sprite List** (lower right: choose who you're coding). *"Four zones. Learn these and you've learned the editor."*
- 2 **"Point and Name" game (10m).** See the box below. It turns the tour into a game so nobody has to sit through a lecture.
- 3 **Green flag & red stop (2m).** Flag = run every **when**  **clicked** script. Stop = halt everything. "Your child will press stop a lot, and that's healthy experimenting." Show the small / large / full-screen stage buttons.
- 4 **The #1 beginner trap (2m).** Live: add the Scratch-library "Dog2" (or any) sprite, select it, and show that the cat's code seems to "vanish". *"Code belongs to whichever sprite is selected. Nine times out of ten, 'my code isn't working' means the wrong sprite is selected."*
- 5 — Break (10m) —
- 6 **Code / Costumes / Sounds tabs (2m).** Click the cat's Costumes tab: two walking costumes. "Switching costumes is how animation works. We'll use that in Module 2."
- 7 **Saving, for real (4m).** Everyone does **File → Save to your computer**, finds the `.sb3` in Downloads, then **File → Load from your computer** to reopen it. Anyone signed in also uses **File → Save now**. Point out that the offline Scratch app uses the same File menu, so files move freely between the app and the website.
- 8 **Kid-account walkthrough (2m, on screen only).** Click **Join Scratch** and walk through it without submitting: username that doesn't reveal the real name, password you both can find, birth month/year, **parent/guardian email**, then the confirmation email. "Do this at home *with* your child."

"POINT AND NAME" GAME (10 MIN)

Round 1: you call, they point (3m). Call out a name and everyone points at the screen, finger on the glass. Go faster as you go: "Stage!" "Blocks palette!" "Green flag!" "Sprite list!" "The Looks category!" "The Costumes tab!" "The stop button!" "Where's the cat's name?" You can see at a glance who's lost, so walk over.

Round 2: you point, they name (3m). Point at zones on the projector and the room calls out the name. Then point at a colored dot and have them call the category *and* say one block it holds.

Round 3: in pairs, find it (4m). Partners take turns challenging each other: "Find a block that plays a sound." "Find the block that starts a program." (gold Events → **when** ► **clicked**) "How many colors of dot are there?" (nine, including the pink My Blocks.) Whoever finds it drags it into the code area and clicks it once to see what it does.

Keep it light and competitive. When they're done, say: *"That's exactly the game you'll play with your child."*

◆ LAB — SAVE & RELOAD DRILL

No build file. Everyone leaves the break with a project saved as a **.sb3** (or saved to their account) and has reopened it once. This habit matters more than any block.

Circulate for: files saved somewhere the person can't find again (walk them to Downloads); a shared library computer that wipes files on logout (have them email the **.sb3** to themselves); account sign-ups waiting on an unconfirmed email.

◆ TEACH-IT ROLE-PLAY — "THE 60-SECOND TOUR" (5 MIN)

Now at the screen. Partner A is the child and holds the mouse; Partner B gives the tour **with hands in their lap**. Swap once (2m each), then debrief for 1 minute.

KID CARD B — LEO, 6

Mood: Excited, wiggly. **Typical behavior:** Can't read most of the block words yet. Clicks everything fast, including the cat, and loses interest in anything longer than a sentence. Loves pressing big buttons.

SAY: *"See these four parts? Over here are the blocks. That's your toy box. In the middle is where you build. Up here is where the cat does what you say. And this green flag makes it GO. Want to press it?"*

SAY: *"How many colors of block can you find? Let's count the dots."*

SAY: *"Can you teach me the four parts?" (older kids)*

Debrief question: "Coaches, did your hand go for the mouse? How many times?" Name the rule: **hands in your lap**. Point with a finger, ask a question, let them click. For a 6-year-old, use nicknames ("toy box", "GO button") and skip the vocabulary.

◆ WATCH - OUTS

- ◆ **Code on the wrong sprite.** This is the classic mistake. From today on, it's the first thing to check whenever something "doesn't work".
- ◆ **"I deleted everything!"** Usually the wrong sprite is selected. Otherwise, right-click the code area → **Undo**, or **Edit → Restore Sprite** for a deleted sprite. Show this calmly, once.
- ◆ **Browser zoom and small text.** Adults squint at the tiny block text. Teach **Ctrl/⌘ +** to zoom the page, and the **+ / -** magnifier buttons at the bottom-right of the code area.
- ◆ **The language globe.** Someone will click it and switch to another language. It's the globe icon at top-left; switch back from there.
- ◆ **Home-device questions** ("Will this run on our old iPad?"). Scratch runs in any modern browser, but a tablet's touch drag is fiddly and there's no app for iPad (use the browser). Offer to answer at the break so the room keeps moving.
- ◆ **Mixing up the website and the app.** The editor is the same in both. Only saving differs: an account lives on the website; the app saves files.

◆ CHECK - QUIZ

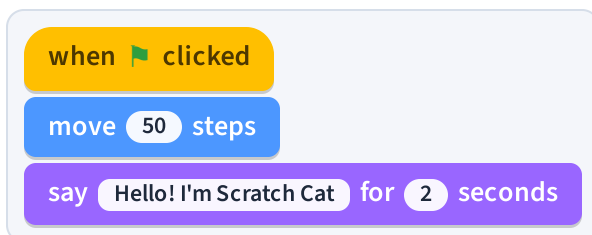
Answers: nothing happens when you run it → **the code is on a different sprite from the one selected** (B); the button that runs the project is **the green flag** (A); category colors help you **find the kind of block you need in the palette** (B); *new*: without an account, you keep your work with **File → Save to your computer** (C).

◆ OBJECTIVES

- ◆ Drag, drop and snap blocks together to build a script
- ◆ Start a program with the **when**  **clicked** event
- ◆ Combine Motion and Looks blocks to make a sprite act
- ◆ Test, tweak and troubleshoot a simple project
- ◆ Coach their child through building it themselves

◆ TEACHING SEQUENCE – LIVE DEMO SCRIPT

- 1 **Show the destination (1m).** Run the finished program from the reference file. *"Three blocks. That's a complete, working program. Let's build it one block at a time."* Everyone opens a fresh project with the cat **selected** in the sprite list.
- 2 **The hat (1m).** *"Gold Events. Drag **when**  **clicked** into the middle. It's called a hat because it sits on top and starts everything."*
- 3 **Move (2m).** *"Blue Motion. Drag **move 10 steps** under the hat. Watch for the grey shadow, and let go when you see it."* Deliberately drop one a little too far away and show that the flag does nothing. Then snap it and click the flag: the cat moves a tiny bit.
- 4 **Say (2m).** *"Purple Looks. Drag **say Hello! for 2 seconds** under the move block."* Click the white oval and type **Hello! I'm Scratch Cat**. Click the **10** and change it to **50**. Run it.
- 5 **Sequence (2m).** *"Scratch runs blocks top to bottom, one after another. That's called a sequence. Swap the order and the cat talks first, then walks."* Drag the say block above move and run it to prove it. Then show the reset: add **go to x: 0 y: 0** right after the hat so the cat stops marching off the edge.



Build the three-block script, run it, then pick "Make it yours" challenges from the workbook: change the message to a joke; change 50 to 200; add a second **say**; **start sound Meow**; **change size by 25**; **turn 15 degrees**; **next costume** for a step of walking. Finish by saving. Reference: `labs/module-1/l1-3-cat-move-talk.sb3`.

Circulate for: blocks that *look* attached but aren't (drag the hat and see whether everything comes with it); the cat piling up at the right edge; typing into the wrong oval; a participant who's done in 3 minutes (give them the "walk *and* look like walking" challenge, then make them a table helper).

◆ TEACH-IT ROLE-PLAY – "TREASURE HUNT, NOT DICTATION" (5 MIN)

Partner A is the child and drives; Partner B coaches the build using only **questions**, with no direct instructions like "drag the move block". Swap once, then debrief for 1 minute.

KID CARD C — SAM, 11

Mood: Impatient and confident. **Typical behavior:** "Can we just make a game?" Clicks ahead before you finish talking. Types silly messages and tests the limits. Goes quiet and says "this is dumb" when a block won't snap.

SAY: *"We want the cat to walk. Which color do you think has the moving blocks?"*

SAY: *"The message is yours. What should the cat say?"*

SAY: *"Hmm, it didn't work yet. Interesting! Why do you think it did that?"*

SAY: *"Can you make the cat cross the stage and say two things?" (goal, not steps, for age 12+)*

Debrief question: "What did the coach do when the block wouldn't snap?" Praise any coach who stayed calm and asked a question. Remind them: **the words, the number and the green flag belong to the child**. End on a high: they show the project to someone.

◆ WATCH-OUTS

- ◆ **Trackpad drag.** Holding a click while moving is hard on a trackpad, so blocks drop in the wrong place. Hand over a mouse. On a trackpad, suggest a firm press-and-hold, or three-finger drag on a Mac.
- ◆ **Clicking a block runs it.** A single click on any block or stack runs it once, even without the flag. That's handy for testing, but it confuses people who expected the flag only.
- ◆ **Cat walks off-screen.** Every run adds 50 more steps. Fix it by dragging the cat back, or with **go to x: 0 y: 0** after the hat.
- ◆ **Upside-down cat** after the turn challenge. Set rotation style to left-right in the sprite info (direction). It's covered properly later, so just fix it for them.

- ◆ **say vs say ... for 2 seconds.** The plain **say** block leaves the speech bubble up. The lesson uses the timed version.
- ◆ **"I'm so slow."** Say it plainly to the room: dragging feels clumsy for the first hour for everyone, and by Lesson 2.2 it's automatic.

◆ CHECK — QUIZ

Answers: top of the script → **when**  **clicked** (B); a stack runs **top to bottom, in order** (A); the cat moves but won't talk → **is the "say" block snapped onto the stack?** (B); *new*: swap move and say and the cat **talks first, then walks** (C).

CLOSING THE SESSION

S1 WRAP-UP (10 MIN)

Journal (5m). Workbook journal prompts for 1.3, plus the confidence number from 1–10. Tell them to write it down, because they'll compare it in Session 6. **Share (2m).** Two or three people read one line. **Try at home this week (2m):** (1) have the "want to make your own game?" chat; (2) let your child build the cat program, hands in your lap; (3) give them a "coder's notebook"; (4) optional: create a kid account together and confirm the email. **Last minute:** everyone checks their `.sb3` is saved where they can find it. Preview S2: "Next time the cat stops doing one thing and starts *behaving*, with timing, loops and choices."

Module 2 — Core Coding Concepts

How to teach the three ideas under every program ever written: sequences, loops, and conditionals. Each one starts away from the screen, with a game the adults can later play at home. By the end, parents have a walking cat, a cat they drive with the arrow keys, and words for explaining both to a child.

3 lessons

~90 min teaching

Lessons 2.1–2.3

Level Beginner

Maps to Session S2

HOW TO USE THIS GUIDE

- Every lesson follows the same shape: **Unplugged** (the idea off the screen) → **Teach** (a short live demo) → **Lab** (they build it) → **Teach-It** (pairs role-play teaching a child) → **Check**. Module 2 leans hardest on the unplugged part. The two off-screen games are what parents take home, so give them their full time.
- Project the editor zoomed in.** Use the + magnifier at the bottom-right of the code area (and browser zoom) so the back row can read block text. Tell the room they can zoom too; many will need to and won't ask.
- Have mice ready.** Nesting blocks inside the "mouth" of a C-shaped loop or *if* is the hardest drag in the course so far. On a trackpad it can feel impossible. Put a basket of USB mice on the front table.
- Red stop sign, early and often.** Before the first *forever*, show that the red stop above the stage ends everything. People who are afraid of breaking the computer relax once they've seen it.
- Teal** = hands-on, **amber** = watch-outs, **purple** = concept / Teach-It.

THE THROUGH-LINE

One cat, three upgrades. It **performs** a timed greeting (sequence), **walks** across the stage with moving legs (loop), then **obeys the arrow keys** (forever + if). Name the pattern at the end of the session: "Steps in order, steps that repeat, steps that choose. Those three ideas are in every app on your phone." The arrow-key script comes back in Module 3's games, so ask everyone to save it.

MODULE AT A GLANCE

LESSON	TOPIC	TIME	PARENTS LEAVE ABLE TO...
2.1	Sequences & Timing	33m	Explain that order changes behavior; pace a routine with <i>wait</i> ; run the robot game

LESSON	TOPIC	TIME	PARENTS LEAVE ABLE TO...
2.2	Loops — Making Things Repeat	27m	Choose <i>repeat</i> vs <i>forever</i> ; put the right blocks inside the C; animate a walk
2.3	Conditionals & Sensing	30m	Build <i>forever</i> + <i>if</i> checks; read a condition aloud; drive the cat with arrow keys
—	Check-in · break · wrap-up	30m	Journal share, 10-min break, quick-check round and "try at home"

SESSION S2 RUN-SHEET (120 MIN)

CLOCK	MIN	SEGMENT	WHAT HAPPENS
0:00	10	Welcome & check-in	Phones on silent. Two or three volunteers read one line from their 1.3 journal. Ask: "Did anyone show the moving cat to a child? What did they say?" Make sure everyone can open their S1 project.
0:10	10	Unplugged: Program the Facilitator	The room gives you instructions to walk to the door. You follow them literally. This brings out sequence, order, and loops (see 2.1).
0:20	8	2.1 Teach	Live demo: swap move/say order; knock-knock joke with and without <i>wait</i> .
0:28	10	2.1 Lab	Greeting routine (12-1-greeting-routine.sb3).
0:38	5	2.1 Teach-It	Pairs: the robot game with a kid card.
0:43	10	2.2 Teach	The hard way vs the loop way; <i>repeat</i> vs <i>forever</i> ; inside vs outside the C.
0:53	10	Break	Leave the projector on the walking-cat script so early returners can start.
1:03	12	2.2 Lab	Walking cat (12-2-walking-cat.sb3). Fast finishers: the forever-patrol challenge.
1:15	5	2.2 Teach-It	Pairs: spot-the-loop + the "is there a block that could do all these?" question.
1:20	5	Unplugged: "If it's raining, then..."	Everyday-conditionals call-and-response (see 2.3).
1:25	8	2.3 Teach	if/then, Sensing hexagons, the forever + if game loop, if/then/else.
1:33	12	2.3 Lab	Arrow-key cat (12-3-arrow-key-cat.sb3).
1:45	5	2.3 Teach-It	Pairs: coach the arrow-key build using questions only.

CLOCK	MIN	SEGMENT	WHAT HAPPENS
1:50	10	Wrap-up	Quick-check round (hands up A/B/C for 2–3 questions), 3-minute journal, "try at home this week," preview Module 3.
120			Saturday half-day format: S1 + S2 run back to back, so this becomes the second half of Day 1. Use the lunch break as the "break" row and keep everything else as written.

IF YOU'RE RUNNING BEHIND

Protect the two unplugged games and the 2.3 lab. They carry the module. Cut in this order: the 2.2 forever-patrol challenge (make it homework), then the 2.1 lab challenge (sound + costume), then trim each Teach-It to a single round. Never cut the wrap-up journal. That habit is how parents keep going between sessions.

2.1 Sequences & Timing

33 min

◆ OBJECTIVES

- ◆ Explain a sequence as steps carried out in order, top to bottom, followed literally
- ◆ Show that the same blocks in a different order behave differently
- ◆ Use **wait (1) seconds** (Control) for pacing, and know that **say ... for 2 seconds** already pauses
- ◆ Run the "robot game" with a child to teach sequencing away from the screen

◆ TEACHING SEQUENCE

- 1 **Unplugged warm-up: Program the Facilitator (10m).** Stand on the far side of the room from the door. Say: **SAY:** *"I'm a robot. I can only do exactly what you tell me, one instruction at a time. Your job: get me out that door."* Follow every instruction *literally*. "Walk to the door" → "I don't know 'walk to'." Give me steps." "Take a step" → one tiny shuffle. "Turn" → "Which way? How far?" Let them run into your literalness a few times, then write their instructions on the whiteboard as a numbered list. Draw out three things as they come up:
 - **Order matters.** Swap "turn right" and "take 3 steps" on the board and walk the new version into a wall.
 - **Precision matters.** "Turn" needs a direction and an amount, the way *turn 15 degrees* has a number in it.
 - **The loop seed.** Someone will say "step, step, step, step..." Stop and ask: **SAY:** *"Is there a shorter way to say that?"* Someone offers "take 8 steps" or "keep walking until you reach the door." Write it down and circle it: "Hold that thought. That's next hour's lesson." End by opening the door and asking what went wrong along the way. Every answer is a bug they found and fixed. That was debugging.
- 2 **Order changes everything (4m).** Live demo on a fresh project: *when green flag clicked → move (100) steps → say (Made it!) for (2) seconds*. Run it. Drag the say block above the move. Run again. **SAY:** *"Same blocks, different order, different show. The computer did what we said, not what we meant. Nothing broke. It showed us exactly what our recipe says."*
- 3 **The wait block (4m).** Open the orange Control category; *wait (1) seconds* is at the top. Build the knock-knock joke without waits, run it, then add *wait (1) seconds* between lines. Point at the words "for 2 seconds" on the say block. That block holds by itself. Plain say doesn't. Launch the lab.
- 4 **Lab (10m) then Teach-It role-play (5m), both below.**

A four-beat welcome on the cat: *when green flag clicked* → *go to x: (0) y: (0)* → *say (Hi, I'm Scratchy!) for (2) seconds* → *wait (1) seconds* → *move (120) steps* → *say (Welcome to my world!) for (2) seconds*. Challenge: add *start sound (Meow)* and *next costume* between the wait and the move. Reference: [labs/module-2/l2-1-greeting-routine.sb3](#).

Circulate for: blocks that *look* stacked but aren't snapped (a gap means that block never runs, so drag it closer until it clicks); a missing *go to x: 0 y: 0* (the cat walks off the stage after two runs); leftover S1 scripts that also start on the green flag and fight the new one.

◆ TEACH-IT ROLE-PLAY – THE ROBOT GAME

Pairs. One partner is the child and one is the parent. Hand the "child" the kid card. The parent asks the child to "program" them to make a peanut-butter sandwich (mime it), then follows each step hilariously literally. Swap roles for the last minute.

KID CARD — MAYA, AGE 6

Giggly, loves being in charge. Can't read "seconds" yet. Gives huge instructions ("make the sandwich!") and gets frustrated when the robot doesn't "just know." Wants to press the green flag again and again.

SAY: *I'm a robot. I only do exactly what you say, one step at a time. What's my first step?*

SAY: *You said "put peanut butter on the bread," so I put the whole jar on the bread! How can we change that step?*

SAY: *The computer is a robot just like I was. If it looks wrong, let's read our steps in order and find the mix-up.*

Debrief (30s): "What did the parent do when Maya got frustrated?" The answer to listen for: *laughed with her, then shrank the step*. Nobody said she was wrong.

◆ WATCH-OUTS

- ◆ **"My cat is frozen!"** It's almost always a *say ... for 2 seconds* holding, as designed. Read the block aloud with them. Don't let anyone restart their laptop over it.
- ◆ **Dragging from the middle of a stack.** Grabbing a block takes *every block below it* along with it. That's a feature, but it surprises adults. Demo it once on purpose.
- ◆ **Deleting by accident.** A block dropped back onto the palette disappears. **Ctrl/Cmd + Z** (or right-click the code area) brings it back. Say so before the lab.
- ◆ **Self-comparison.** Faster neighbours finish the challenge while others are still snapping block 3. Say out loud: **SAY:** *"The challenge is optional. Everyone's goal today is the six blocks."*
- ◆ **"The robot game is babyish."** Some adults feel silly playing it. Remind them they're rehearsing a game for their kid, not being tested on it. For older kids at home, flip the roles: the child is the robot and

catches the adult's bugs.

◆ CHECK – QUIZ

Answers: a sequence = **A) steps carried out in order, one after another**; *wait* lives in **B) Control (orange)**; cat "freezes" 2 seconds = **B) a say-for-2-seconds or wait block holding, as designed**; *say ... for 2 seconds* vs plain *say* = **B) the timed one pauses the script before the next block; plain say moves straight on.**

◆ OBJECTIVES

- ◆ Use **repeat (10)** and **forever** (Control) and choose between them
- ◆ Explain that only the blocks inside the C-shape repeat
- ◆ Build a walking animation with *move* + *next costume* + *wait* inside a loop
- ◆ Stop a runaway *forever* with the red stop sign

◆ TEACHING SEQUENCE

- 1 **The hard way first (3m).** Point back at the circled line from the warm-up ("step, step, step..."). Drag in four *move (10) steps* blocks and pretend to reach for sixteen more. Groan. Then swan in *repeat (20)* with one move inside. **SAY:** "One loop replaces twenty blocks. And changing 20 to 100 takes one second."
- 2 **repeat vs forever (4m).** Both are orange Control blocks shaped like a C. *repeat* = you know how many times. *forever* = until someone presses stop. Put a *say (Done!)* block *after* a forever loop and show that it never runs. Now press the red stop sign. **SAY:** "Forever means forever. The stop sign is always your way out, and you can't break anything with it."
- 3 **Inside vs outside the C (3m).** Build the two comparison scripts: *repeat 3 { say Ha! }* vs *repeat 3 { move 10 } then say Done!*. Run both. Point at the mouth of the C: "Inside repeats, outside runs once." Show how to drop a block into the mouth so the C stretches to fit. Launch the lab.
- 4 **Lab (12m, after the break) then Teach-It role-play (5m).**

◆ LAB — A REAL WALKING CAT

when green flag clicked → go to x: (-200) y: (0) → repeat (20) { move (20) steps · next costume · wait (0.1) seconds }.

The cat strides across with moving legs. Let them tune the wait to feel speed change. Challenge: wrap the walk in *forever*, add *if on edge*, *bounce* and *set rotation style [left-right]* so it patrols back and forth. Reference: [labs/module-2/l2-2-walking-cat.sb3](#).

Circulate for: *next costume* or *wait* sitting *below* the loop instead of inside it (the cat glides with frozen legs, or zooms across in a blink); a missing *go to* start position; upside-down cats in the challenge (rotation style).

◆ TEACH-IT ROLE-PLAY — SPOT THE LOOP

Round 1 (2m): the "parent" plays spot-the-loop out loud. Round 2 (3m): the "child" has built a script with the same block copied many times. The parent may only ask questions and must not fix anything.

KID CARD — LEO, AGE 9

Proud of his work. He's copy-pasted *move (10) steps* fifteen times and it works. He bristles at "that's wrong." He likes counting and races.

SAY: *Brushing teeth is a loop: scrub, scrub, scrub, repeat until clean! Can you think of another loop?*

SAY: *That works! Is there a block that could do all of those for us at once, so we don't have to make so many?*

SAY: *Is your block inside the mouth of the C, or outside?*

Debrief: "Did Leo find the loop himself?" If the parent ended up dragging the block in for him, that's normal for a first try. Name it kindly and point ahead to 5.1, "The Art of Not Taking Over."

◆ WATCH - OUTS

- ◆ **Trackpad nesting.** Dropping a block *into* the C mouth is a precise drag. If someone's block keeps landing below the loop, hand them a mouse before they conclude they're "not a computer person."
- ◆ **Fear of forever.** Some adults won't press the green flag on a forever script. Stand beside them, click the red stop together once, and move on.
- ◆ **"The cat zooms across in a blink."** The *wait* is missing or sits outside the loop. Put *wait (0.1)* inside and try 0.2 or 0.3.
- ◆ **Upside-down cat** (challenge). By default a sprite rotates all the way around. Add *set rotation style [left-right]* at the top.
- ◆ **Break-time drift.** Phones come out at the break and family logistics take over. Restart the lab on time with the script already on the projector. Latecomers can copy it without asking.

◆ CHECK — QUIZ

Answers: blink exactly 5 times = **A) repeat 5**; only **B) the blocks inside the C-shape** repeat; a block after *forever* will **A) never run**; cat zooms across in a blink = **A) add or lengthen a wait inside the loop**.

◆ OBJECTIVES

- ◆ Use **if <> then** and **if <> then / else** (Control) to make decisions
- ◆ Read the hexagon Sensing blocks: *touching (mouse-pointer)?*, *key (space) pressed?*, *mouse down?*, *touching color?*
- ◆ Combine **forever + if**, the "game loop," so a check runs constantly
- ◆ Teach if/then through everyday rules and a drive-the-cat game

◆ TEACHING SEQUENCE

- 1 **Unplugged: "If it's raining, then..." (5m).** Call-and-response. You say the "if" the room shouts the "then": **SAY:** "If it's raining..." (umbrella!) "If the light is red..." (stop!) "If the baby is asleep..." (whisper!) Then add "otherwise": "If it's raining, THEN umbrella, OTHERWISE..." (sunglasses!). Now each pair writes two rules from their own household on a sticky note: one if/then and one if/then/otherwise. Collect three or four and read them aloud. Finish with the point that matters: **SAY:** "Your kids already live by conditionals. Bedtime, screen time, dessert. Today we just give them the block." One follow-up question sets up the game loop: "How often do you check whether it's raining? Once in the morning, or all day?" The answer is *all day, constantly*. Hold on to that.
- 2 **if/then + Sensing (3m).** Show *if <touching (edge)?> then { say (Ouch!) for (1) seconds }*. Point at the pointy hexagon slot: it only accepts a true/false question, and those come mostly from the light-blue Sensing category. Read it aloud as a sentence.
- 3 **forever + if, the game loop (3m).** Build *when green flag clicked → forever { if <touching (mouse-pointer)?> then { start sound (Meow) } }*. Hover over the cat and it meows. Now pull the if *out* of the forever and run it again: it checks once at the start and never again. **SAY:** "Remember the rain question? All day. That's what forever does. It keeps asking."
- 4 **if/then/else (2m).** Two mouths: top when true, bottom when false. Read it aloud with "otherwise." Kids find that friendlier than "else." Launch the lab.
- 5 **Lab (12m) then Teach-It role-play (5m).**

◆ LAB — DRIVE THE CAT WITH ARROW KEYS

when green flag clicked → forever { four if/then blocks: key (right arrow) pressed? → change x by (10); left arrow → change x by (-10); up arrow → change y by (10); down arrow → change y by (-10) }. Suggest building one arrow, testing it, then right-click → **Duplicate** the if block and change the dropdown and number. Challenge: "if touching (edge)?, say Ouch!" Reference: [labs/module-2/12-3-arrow-key-cat.sb3](#).

Circulate for: an *if* sitting below the forever instead of inside it; *change x* vs *change y* mixed up (left/right = x, up/down = y); the "wrong" sign on a number (-10 is left/down); stage without keyboard focus (they have to click the stage once).

◆ TEACH-IT ROLE-PLAY – COACH WITH QUESTIONS

The "child" drives the mouse and keyboard. The "parent" keeps their hands in their lap and may only ask questions. Goal: one working arrow key.

KID CARD — SAM, AGE 12

A little too cool for this: "Scratch is for little kids." Wants to make "a real game." Types fast, skips reading, and gets annoyed when the cat doesn't move. He's forgotten to click the stage.

SAY: *How does the cat know you pressed a key? Which color has "senses"?*

SAY: *We want it to check all the time, not just once. What loop does that?*

SAY: *This forever-plus-if pattern is how real games work. Every game you play is checking things like this many times a second.*

Debrief: "What made Sam lean in?" Usually it's hearing it connected to real games. For younger children (5–7), the lesson says a single *if* is plenty: "if touching the pointer, meow."

◆ WATCH-OUTS

- ◆ **"My cat won't move!"** Check keyboard focus first: **click the stage once**. Next, check the nesting: change → inside if → inside forever.
- ◆ **Laptop arrow keys.** Some small keyboards have tiny or shared arrow keys (Fn). If someone struggles to press them, let them pick other keys in the dropdown (for example w/a/s/d).
- ◆ **Three levels of nesting.** Blocks inside blocks inside blocks is the first real "hold it in your head" moment, and it feels clumsy for almost everyone. Say so before they start. Praise the persistence out loud.
- ◆ **Home-device questions** ("will this work on my kid's Chromebook / iPad?"). Park them on a "parking lot" sheet and answer at the break or after the session. Don't derail the lab.
- ◆ **Operators temptation.** A keen participant asks about *and/or*. That's great for 12+, but save it. Operators arrive naturally in Lesson 3.2.

◆ CHECK – QUIZ

Answers: what fits an "if ____ then" slot = **A) a hexagon (true/false) block**; games wrap ifs in forever **B) so the condition is checked constantly, not just once**; if/then/else is best when **B) you want one thing if true and a different thing otherwise**; correct script but cat won't move = **A) click the stage so it has keyboard focus**.

CLOSING THE MODULE

MODULE 2 WRAP-UP (10 MIN)

Quick-check round (3m): read 2–3 quiz questions and have the room vote A/B/C with fingers. **Journal (4m):** the 2.3 "progress & feelings" prompt: how confident do you feel compared with Session 1? **Try at home this week (2m):** play the robot game and a round of if/then rules at dinner or in the car, and show your child the arrow-key cat. Let them change the numbers. **Preview (1m):** "Sequences, loops, conditionals: you now know the three ideas under all programming. Next session we put them together into a story and a real game with a score."

Module 3 — Building Real Projects

How to run the session where parents stop following steps and start building things their kids will want to show off: a two-character animated story, a real catch game with a score, and a calm method for fixing bugs, practised on a game full of planted bugs.

3 lessons

2 hrs in class

Lessons 3.1–3.3

Level Beginner

Maps to Session S3

HOW TO USE THIS GUIDE

- Same rhythm for every lesson: **Frame** → **Demo** → **Lab** → **Teach-It role-play** → **Check**. The run-sheet below is the clock; the lesson blocks are the content.
- This is the busiest session.** Three lessons fit into two hours only if you demo tight and send people to the labs quickly. The reference `.sb3` files in `labs/module-3/` are your safety net. A participant who falls behind on 3.2 loads the finished game and still does the Bug Hunt.
- The emotional arc matters.** 3.1 feels delightful, 3.2 feels hard (say so out loud: "this one is a step up"), and 3.3 turns that difficulty into a skill. Parents who struggled in 3.2 often shine in the Bug Hunt.
- 3-Saturday format:** this session is the morning block of Saturday 2 (S4 follows after lunch). Keep the same timings.
- Teal** = hands-on, **amber** = watch-outs, dashed boxes = lines to say out loud.

THE THROUGH-LINE

Three projects, one arc: **make it** → **make it a game** → **make it work**. The story (3.1) teaches sprites to cooperate with *broadcasts*. The catch game (3.2) adds a *variable* and combines every block category so far. The Bug Hunt (3.3) hands back that same game with four planted bugs, so participants debug code they already understand. The Teach-It thread matches it: *story before code, design before code, ask, don't fix*.

BEFORE THE SESSION

- Copy `labs/module-3/` to every machine (or a shared drive / USB): `l3-1-conversation.sb3`, `l3-2-catch-the-apple.sb3`, `l3-3-bug-hunt-starter.sb3`. Test loading one with **File** → **Load from your computer**.
- Print the three **kid cards** (in each lesson block below) and one **4-step debugging card** per pair (Watch → Predict → Change ONE thing → Test).

- ◆ Bring mice, a few sticky notes (for the "SCORE" whiteboard demo in 3.2), and a stuffed animal for the rubber-duck demo in 3.3. It gets a laugh and people remember it.
- ◆ Pair people up in advance: the Bug Hunt runs in **debugging-buddy pairs**. Mix confident and cautious participants.

MODULE AT A GLANCE

LESSON	TOPIC	TIME	PARTICIPANTS LEAVE ABLE TO...
3.1	An Interactive Animated Story	28m	Stage a two-sprite conversation with <code>broadcast</code> / <code>when I receive</code>
3.2	A Simple Game with Variables & Score	32m	Create a <code>score</code> variable and build a playable catch game
3.3	Debugging & Remixing	30m	Find and fix bugs with the 4-step method; coach a frustrated child
—	Warm-up · break · wrap-up	30m	Journal share, 10-minute break, "try at home this week"

SESSION S3 RUN-SHEET

CLOCK	MIN	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up & check-in	Journal share from S2 (one "what clicked" per table). Ask: "Who tried the arrow-key cat at home?" Phones on silent. Mice plugged in.
0:10	12	3.1 Demo	x/y grid, <code>go to</code> vs <code>glide</code> , broadcast "stage director" demo
0:22	10	3.1 Lab	Two-character conversation (broadcast chain)
0:32	6	3.1 Teach-It	Role-play: "Story before code" (kid card: Maya, 6)
0:38	12	3.2 Demo	Variable = whiteboard; make <code>score</code> ; game plan; build the apple live
0:50	10	Break	Leave the game plan flowchart on screen
1:00	15	3.2 Lab	Catch the Apple game with a live score
1:15	5	3.2 Teach-It	Role-play: "Design first" and the scope trap (kid card: Theo, 10)
1:20	8	3.3 Demo	Debugging mindset, 4-step method, usual suspects, See Inside & remix
1:28	17	3.3 Lab	Bug Hunt in debugging-buddy pairs (4 planted bugs)
1:45	5	3.3 Teach-It	Role-play: "Ask, don't fix" (kid card: Priya, 9)

CLOCK	MIN	SEGMENT	WHAT HAPPENS
1:50	10	Wrap-up	Journal (5 min), Bug Hall of Fame shout-outs, try-at-home list, preview S4 (clones) and the Game Design Card homework
120 Total: 10 + 12 + 10 + 6 + 12 + 10 + 15 + 5 + 8 + 17 + 5 + 10			

IF YOU RUN LONG

Protect the Bug Hunt. Cut in this order: (1) skip the 3.1 backdrop-change challenge, (2) shorten the 3.2 lab by letting slower pairs load `l3-2-catch-the-apple.sb3` and just play it, (3) demo See Inside for 1 minute instead of 3. Never cut the wrap-up journal. It's how adults consolidate.

3.1 An Interactive Animated Story

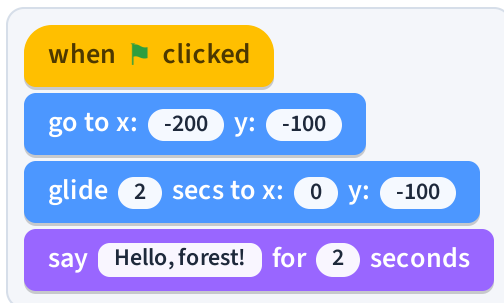
28 min

◆ OBJECTIVES

- ◆ Add multiple sprites and a backdrop from the libraries
- ◆ Position sprites with `go to x: y:` and `glide _ secs to x: y:`
- ◆ Coordinate sprites with `broadcast` and `when I receive`
- ◆ Coach a child to plan the story out loud before coding it

◆ TEACHING SEQUENCE

- 1 **Set the scene (2m).** New project → **Choose a Backdrop** (bottom-right, under the stage) → "Forest."
Choose a Sprite → add a Bear. Remind them: *click the sprite first*, then add blocks.
- 2 **The grid, teleport vs slide (4m).** Stage is x -240...240, y -180...180, centre 0,0. Build the entrance script live:



SAY: Watch the numbers under the stage change as I drag the cat. That's where x and y come from. You never have to guess them.

- 3 **The problem broadcasts solve (6m).** Ask the room: "How does the Bear know the Cat has finished talking?" Let them suggest `wait` blocks, then point out that the timing drifts. Act it out: you are the stage director, two volunteers are Cat and Bear. Say "Cue: your turn!" and the Bear answers. Then build it: Cat gets `broadcast [your turn]` (choose **New message** and name it), Bear gets `when I receive [your turn]`.

SAY: A broadcast is an invisible shout. Any sprite can shout it, and any sprite that's listening springs into action.

- 4 **Lab launch (0m, just go).** Show the 4-line plan on the slide and send them.

◆ LAB – A CONVERSATION THAT PLAYS ITSELF

Chain four lines: Cat (green flag) says a line, then `broadcast [line 2]` → Bear (`when I receive [line 2]`) replies, then `broadcast [line 3]` → Cat replies, then `broadcast [line 4]` → both glide off. Reference: `labs/module-3/l3-1-conversation.sb3`. Fast finishers: the Stage's `when I receive [night]` → `switch backdrop to` challenge.

Circulate for: only *one* script starts with the green flag; blocks on the wrong sprite; `broadcast` used where `when I receive` was meant (both are gold Events blocks); messages with near-identical names ("line2" vs "line 2").

◆ TEACH-IT ROLE-PLAY – STORY BEFORE CODE (6 MIN)

KID CARD • MAYA, AGE 6

Bouncy, excited, wants to spend ten minutes scrolling the sprite library. When asked "what does the character say?" she shrugs: "I don't know." Loves doing silly voices. Gets bored if the grown-up types for more than a minute.

Pairs: one plays Maya, one coaches through the 3-step story warm-up: **Who?** (two characters with names) → **Where?** (backdrop) → **What happens?** (act it out in voices). Swap once if time allows.

SAY: Let's be the characters first! You be the bear, I'll be the cat. What does the cat say? ...Great, now let's make Scratch say exactly that.

SAY: Pick any two you like. We can come back to the library later. Who's our hero today?

Debrief (1 line): for ages 5–7, two lines each is plenty. The child picks the words, and the parent wires the broadcasts.

◆ WATCH-OUTS

- ◆ **Both sprites talk at once.** This is the lesson's classic bug: both scripts start with `when clicked`. Only line 1 starts on the flag. Every other line starts with `when I receive`.
- ◆ **"Where did my code go?"** They added blocks with the Bear selected, then clicked the Cat. Reassure them: nothing is lost, each sprite has its own code area.
- ◆ **Trackpad drag struggles** make broadcast dropdowns fiddly. Mice help. So does zooming the browser (Ctrl +) for anyone using reading glasses.
- ◆ **Sprite library rabbit hole.** Adults do it too. Give a 60-second limit on choosing sprites.
- ◆ **Glide ends off-stage** because of typos like `x: 2000`. Show them how to read the x/y under the stage.

◆ CHECK – QUIZ

Answers: sprites take turns via **broadcast and "when I receive"**; smooth movement = **glide _ secs to x/y** ;
both talking at once = **both start with "when 🚩 clicked" instead of "when I receive"**; to change the scene,
give the **Stage** **when I receive [night]** → **switch backdrop to** .

◆ OBJECTIVES

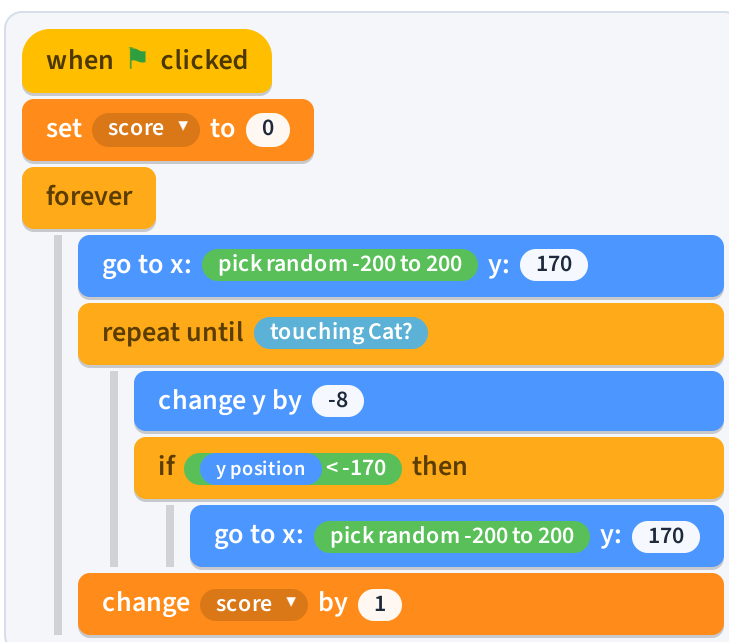
- ◆ Explain a variable and create one (`score` , "For all sprites")
- ◆ Use `set [score] to 0` and `change [score] by 1` ; show the score on the stage
- ◆ Combine loops, `if` , sensing, and operators into a playable game
- ◆ Coach a child to design a game on paper and avoid the scope trap

◆ TEACHING SEQUENCE

- 1 **The whiteboard (3m).** Stick a note on the wall: "SCORE: 0." Pretend to catch something, cross it out, write 1.

SAY: That's a variable: a labeled box the computer remembers. set erases and writes a number. change adds to what's already there.

- 2 **Make the variable (2m).** Variables (orange) → **Make a Variable** → `score` , keep **For all sprites** → OK. Point at the checkbox that shows it live on stage.
- 3 **The game plan (2m).** Show the flowchart slide: apple starts at the top at a random x → falls → touching the cat? yes = +1 → back to top. Say it as a story before any block moves.
- 4 **Build the apple live (4m).** Build it slowly and name each category as you use it (Motion, Control, Sensing, Operators, Variables). The bottom check is the green `<` block from Operators with the `y` `position` reporter (bottom of Motion) dropped into its left slot:



SAY: *Read it as a story: start at the top somewhere random, fall until the cat catches you, jump back to the top if you drop past the bottom, and when you're caught, add a point. Then do it all again, forever.*

5 **Catcher preview (1m).** Show the Cat script on the slide and let them build it themselves in the lab.

◆ LAB – CATCH THE APPLE

Add an Apple sprite, make `score`, build the apple script above, and give the Cat `when clicked` → `forever` → `go to x: (mouse x) y: -140`. Play it. **Save it** (File → Save to your computer). Reference: `labs/module-3/l3-2-catch-the-apple.sb3`. Fast finishers: the 30-second `time left` timer challenge.

Circulate for: `change score by 1` placed *below* the repeat-until's C-mouth (not inside it); `touching` dropdown set to `Cat`, not "mouse-pointer"; `set score to 0` before the forever loop; `mouse x` (Sensing), not typed-in text; the bottom check is `y position < -170`, not `touching edge?`.

◆ TEACH-IT ROLE-PLAY – DESIGN FIRST, AND THE SCOPE TRAP (5 MIN)

KID CARD • THEO, AGE 10

Confident, talks fast, has played a lot of games. Announces he wants to make "Minecraft but with dragons, and fifty levels, and online." Gets impatient with paper: "Can we just start?"

The coach walks Theo through the one-page design (**Goal • Controls • Scoring • Lose condition**) and steers him to one working mechanic, without saying "no."

SAY: *Before we code, tell me your game like you're explaining it to a friend. How do you play? How do you win? Let's write those down. Those are our to-do list.*

SAY: *Dragons are awesome. Let's put fifty levels on the "someday" list and make catching one thing perfect first.*

Debrief: a finished tiny game beats an abandoned huge one, for the child's motivation and the parent's.

◆ WATCH-OUTS

- ◆ **This lesson feels hard, so say so.** Four categories plus a variable is a real step up. Adults compare themselves to the fast finisher at the next table. Tell them the lesson's growth-mindset point: "Big projects feel hard for everyone. Get one piece working, then the next."
- ◆ **Nesting mistakes.** Dropping a block into the wrong C-mouth is the #1 error. Teach them to watch for the white "landing shadow" before releasing the mouse.
- ◆ **Score jumps by many at once:** the apple is still touching the cat for several frames. The `go to` the top at the start of the next loop fixes it. Check that it's there.

- ◆ **"Why not touching edge?"** Someone will swap it in for the bottom check, because it sounds right. The apple starts at y: 170, where it is *already touching the top edge*, so it's sent back to the top every frame and never falls. `y position < -170` means "near the bottom" and nothing else. Don't argue it now. Park it, because it's your live debugging demo in 3.3.
- ◆ **Score never goes up:** the `touching` block names the wrong sprite, or the apple never reaches the cat (check the cat's y is -140 and the apple's bottom check is -170, below it).
- ◆ **Screen-time guilt** sometimes surfaces here ("my kid already plays too many games"). Reframe: making a game is creating, not consuming. Then move on. Don't let it take over the lab.
- ◆ **Unsaved work.** This game is the basis for the Bug Hunt and Module 4. Make everyone save before the role-play.

◆ CHECK – QUIZ

Answers: a variable is a **named box that stores a value the program can change**; add a point with `change score by 1`; `set score to 0` goes **right after the green flag, before the game loop**; to see the score on the stage, **tick the checkbox next to the variable**.

◆ OBJECTIVES

- ◆ Apply the 4-step method: **Watch** → **Predict** → **Change ONE thing** → **Test**
- ◆ Recognize the usual-suspect Scratch bugs and their fixes
- ◆ Use **See Inside** on a shared project; explain remixing and its etiquette
- ◆ Coach a frustrated child through a bug without taking over the mouse

◆ TEACHING SEQUENCE

1 The mindset (2m).

SAY: *The computer isn't wrong and it isn't being mean. It's doing exactly what the blocks say. Debugging is the detective game of finding the gap between what you said and what you meant.*

Swap "It's broken, I failed" for "Interesting, let's find out why."

2 **The 4-step loop (3m).** Show the slide. Stress step 3: *change only one thing*. Then name Scratch's helpers: click a single block to run it, tick a variable's checkbox to watch it, add a temporary `say (score)`, slow things down with `wait`. Demo the rubber duck: explain a script to the stuffed animal and "find" a bug mid-sentence.

3 **Live bug: "Why not touching edge?" (included in the 3m above, or take 1m from See Inside).** In your 3.2 game, swap the bottom check for `if touching edge?` and run it, thinking aloud through the four steps. **Watch:** the apple is frozen at the top. **Predict:** "It's being sent back to the top, so the *if* must be true right away. Is the apple touching an edge at y: 170?" (Yes, the top one.) **Change one thing:** put `y position < -170` back. **Test:** it falls.

SAY: *The block did exactly what I told it: "touching any edge." I meant "the bottom." That gap between what I said and what I meant is the bug.*

4 **Usual suspects + See Inside (3m).** Walk the table on the slide quickly (it's in the workbook). Open any shared project on scratch.mit.edu → **See Inside**. Mention that **Remix** needs a signed-in account and that Scratch credits the original creator automatically.

5 **Bug Hunt launch.** Form debugging-buddy pairs (below) and hand out 4-step cards. Say there are **four** bugs, but not what they are.

◆ LAB — THE BUG HUNT (17 MIN)

Pairs open `labs/module-3/13-3-bug-hunt-starter.sb3` (File → Load from your computer). It is the 3.2 catch game with **four planted bugs**. For each bug they write the symptom, their prediction, and the one

change they made in the workbook's Bug Hall of Fame, then test. Solution: `Labs/module-3/l3-2-catch-the-apple.sb3`.

Circulate for: pairs changing several blocks at once (stop them and ask "which one change are you testing?"); navigators grabbing the mouse; pairs who "fixed" a bug by rebuilding the whole script (fine, but ask them to name the one block that was wrong).

◆ DEBUGGING-BUDDY PAIR FORMAT

- ◆ **Driver:** hands on the mouse. Only the driver touches the computer.
- ◆ **Navigator:** holds the 4-step card and the workbook. Asks the "ask, don't fix" questions: *What did you expect? What happened instead? Which block do you think is responsible? What's one thing we could change to test that?* Writes the Bug Hall of Fame entry.
- ◆ **Swap roles after every bug**, so each person drives twice and navigates twice.
- ◆ **One change, then test.** If a change didn't help, undo it (Ctrl/Cmd+Z or drag it back) before trying the next idea.
- ◆ This is a rehearsal of home: the navigator is practising being the parent who asks instead of grabbing.

◆ THE FOUR PLANTED BUGS & HINT LADDER

Give hints only when a pair asks, one rung at a time, and wait about 2 minutes between rungs. Suggest they hunt in this order (easiest to see first): **1 → 3 → 4 → 2**.

BUG	HINT 1 · WATCH	HINT 2 · PREDICT	HINT 3 · NEARLY THERE
1 Cat drifts strangely	"Move the mouse only left and right. What does the cat do? Now only up and down?"	"The cat's movement lives in the Cat's script. Which block decides where it goes?"	"Look at the round reporter inside <code>go to x:</code> . Is that the mouse's left-right number?"
2 Score carries over	"Play a game, stop, then press the green flag again. What number does the score start at?"	"Something should wipe the whiteboard at the start of each game. Can you find it?"	"The Apple needs <code>set score to 0</code> right after the green flag, before <code>forever</code> ."
3 Apple never falls	"Watch the apple. Is it moving? Tick its y position checkbox. Which way is the number going?"	"Only one block in the Apple's script moves it up or down. Find it."	"Bigger y means <i>up</i> . Look at the number in <code>change y by</code> . Which sign do you need to fall?"
4 Score races while falling	"Watch the score while one apple is still falling. When does it go up?"	"The +1 is running many times. Which loop is it inside?"	" <code>change score by 1</code> should run once, <i>after</i> the apple is caught. Where is it sitting?"

With bug 4 still planted, the score races up even while bug 3 keeps the apple stuck at the top, so pairs often blame the wrong block. That's the point of changing one thing at a time. If a pair "fixes" things by swapping in **touching edge?**, the apple freezes again. Point them back to your live demo. Full fixes with drawn scripts are in the answer key.

◆ TEACH-IT ROLE-PLAY – ASK, DON'T FIX (5 MIN)

KID CARD • PRIYA, AGE 9

Tired after school. Her game "doesn't work" and she has tried the same thing three times. Voice rising: "I'm bad at this!" Keeps pushing the mouse toward the grown-up: "YOU do it."

Same pairs, fresh roles. The coach keeps their hands in their lap and uses the rubber duck, the four questions, and (if needed) a break.

SAY: *Walk me through it like I know nothing. What should this block do? ...And what did it actually do?*

SAY: *The code has a bug; you're a coder who's finding it. Remember the one you squashed last week?*

SAY: *Let's let our brains rest. Coders do this all the time. We'll come back with fresh eyes.*

Debrief: "Who felt the urge to grab the mouse?" (Every hand goes up.) The biggest parent mistake is fixing it yourself. That ends the frustration and the learning at the same time.

◆ WATCH-OUTS

- ◆ **Fear of breaking it.** Some adults won't touch the starter file. Say plainly: "You can't break this. The original is on the drive, and you can reload it any time."
- ◆ **Shotgun debugging.** Pairs change five blocks and "it works now." Praise the result, then ask which change mattered. They should be able to name it.
- ◆ **Loading the wrong file.** Someone opens their own 3.2 game, finds no bugs, and feels clever. Check the title bar says *bug-hunt-starter*.
- ◆ **Remix = cheating?** Parents worry about this. Everything shared on Scratch may be remixed, and credit is automatic. The line to teach is honesty: change something meaningful and keep the credit.
- ◆ **Home-device tech support** ("my kid's Chromebook won't..."). Park it on a sticky note for after class.

◆ CHECK – QUIZ

Answers: golden rule = **change one thing at a time, then test**; See Inside **opens the project's code to explore how it was made**; melting-down child = **have them explain it aloud, or take a short break**; flag clicked and nothing happens = **the code is on the wrong sprite**.

CLOSING THE SESSION

S3 WRAP-UP (10 MIN)

Five minutes of silent journaling (the prompts are in the workbook), then ask two or three pairs to read one Bug Hall of Fame entry aloud and cheer each squashed bug. Recap: **broadcasts let sprites take turns → a variable keeps score → Watch, Predict, Change one thing, Test.**

Try at home this week: quietly move one block in the catch game and let your child debug it (using the four questions, hands in your lap); open one shared project together with See Inside; save the game, because its ideas come back in the Module 4 capstone.

Preview S4: "Next time one apple becomes a whole sky of falling stars with *clones*, and you'll make your own blocks. Homework after S4 is a Game Design Card you fill in *with* your child for Family Build Night."

Module 4 — Leveling Up: Bigger Projects

How to teach clones and My Blocks to the grown-ups in Session 4, then run Session 5, Family Build Night, where parent-and-child pairs plan and build Star Catcher together, from a paper design card to a launch-day showcase.

4 lessons

~4 hrs teaching (2 sessions)

Lessons 4.1–4.4

Level Beginner

Maps to Session S4 + S5

HOW TO USE THIS GUIDE

- This module runs over **two sessions with very different rooms**. S4 is adults only and follows the usual rhythm: **Frame → Teach → Lab → Teach-It role-play → Check**. S5 is **Family Build Night**: children attend, and you mostly stop teaching and start running a room.
- Read the **two run-sheets** first. S5 needs setup the day before (devices, rescue file, printed posters, name tags), so plan it at the end of S4, not the morning of S5.
- Build both reference games yourself before S4, including the rescue file `labs/module-4/l4-3-star-catcher-part1.sb3`. You will demo from them and load them onto devices.
- **Teal** = hands-on, **amber** = watch-outs. **Say:** boxes are lines you can read out loud as written.
- **Half-day format:** in the three-Saturday version, S3 + S4 share Day 2, and S5 + S6 share Day 3. Invite the children for the Day 3 morning only (S5), and send them home before Module 5.

THE THROUGH-LINE

Everything in S4 is a part for S5. The falling stars from 4.1 become Star Catcher's star factory; the **reset game** block from 4.2 becomes the Bowl's setup. Tell the adults this at minute one: "Tonight you learn the parts. Next time you build the machine, with your child as the boss." Seeing the destination makes the abstract ideas (clones especially) worth the struggle.

MODULE AT A GLANCE

LESSON	TOPIC	TIME	PARTICIPANTS LEAVE ABLE TO...
4.1 • S4	Clones — One Sprite, Many Copies	40m	Build a hidden "stamp" that makes falling star clones, and clean the clones up

LESSON	TOPIC	TIME	PARTICIPANTS LEAVE ABLE TO...
4.2 · S4	My Blocks — Make Your Own Blocks	35m	Make a custom block (with and without an input), including reset game
4.3 · S5	Capstone Part 1 — Plan It & Build the Core	45m	Plan a game on a design card, then build the smallest playable Star Catcher with a child directing
4.4 · S5	Capstone Part 2 — Lives, Game Over & Launch Day	40m	Add lives, a Game Over broadcast, tuning, and hold a launch-day showcase

SESSION RUN-SHEET — S4 (ADULTS ONLY)

CLOCK	MIN	SEGMENT	WHAT HAPPENS
0:00	10	Warm-up & check-in	Journal share in pairs: "Which of the four planted bugs from 3.3 did you find first?" Take two answers aloud. Show a 20-second clip of the finished Star Catcher: "This is where we're headed."
0:10	40	Lesson 4.1 — Clones	Teach (15) · Lab: falling stars (17) · Teach-It role-play (6) · Check (2)
0:50	10	Break	Leave the falling stars running on screens. Remind everyone to save first.
1:00	35	Lesson 4.2 — My Blocks	Teach (12) · Lab: jump, dance (times), reset game (15) · Teach-It role-play (6) · Check (2)
1:35	15	Family Build Night briefing	Walk the Game Design Card (workbook, 4.3) using the Star Catcher answers; hand out blank cards. Explain director/typist. Logistics: who's coming, ages, device, sign-in. Hand out the invitation (below).
1:50	10	Wrap-up	Journal (4.1 + 4.2 prompts). Try at home: fill in the Game Design Card <i>with your child</i> , in your child's words; bring it to S5. Collect RSVPs (number of children + ages).
120		Total	

SESSION RUN-SHEET — S5 FAMILY BUILD NIGHT

◆ ROOM SETUP (BEFORE DOORS OPEN — ALLOW 45 MIN)

- ◆ **One device per family**, each with a mouse, the Scratch editor open to a *new* project, and volume at about 30%. Label each table with a family name. Two chairs side by side, child's chair **in front of the mouse**.
- ◆ **Rescue file on every device:** copy `14-3-star-catcher-part1.sb3` to the desktop (or a **Rescue** folder) and test that **File → Load from your computer** finds it. Keep a USB stick with both Module 4 files as backup.

Put `l4-4-star-catcher-final.sb3` only on *your* machine.

- ◆ **Name tags:** child tags say the name big plus **DIRECTOR**; adult tags say **TYPIST**. Kids love the titles, and the tags make the rules visible from across the room.
- ◆ **Wall poster** (large print, at kid eye level, next to the screen): *"The Director decides. The Typist types only when the Director says 'your turn.' The Director presses every green flag."* Add a second poster: *Someday List*, with sticky notes for parked ideas.
- ◆ Projector with the slide deck on the **Family Build Night** title slide. A spare stack of printed blank Game Design Cards and pencils/crayons. A timer you can show on screen.
- ◆ Snack table away from the devices (ask about allergies on the RSVP). A "quiet corner" with paper and crayons for a tired or overwhelmed child, or a younger sibling.

CLOCK	MIN	SEGMENT	WHAT HAPPENS
0:00	10	Arrival & kid-friendly opening	Name tags at the door. Families sit at their device. Play the finished Star Catcher on the projector and invite a child up to try it: "Who's good at catching things? Tonight <i>you</i> make this, and you're the boss."
0:10	5	The rules & the design card	Point at the poster; each pair reads it together. Families with a completed design card put it beside the screen. Families without one get a blank card and fill in just three lines: name, player, how you score.
0:15	45	Lesson 4.3 — Build the core	Steps 1–7, paced from the projector: you show each step (1 min), families build it (4–5 min). Announce "green flag moment!" after every step. Circulate.
1:00	5	Checkpoint: smallest playable version	Everyone plays one round. Families whose game doesn't catch stars yet: save theirs, then load the rescue file (see below). Everyone saves as Star Catcher .
1:05	10	Break	Snacks. Kids leave their games playing and walk around to try other families' games.
1:15	30	Lesson 4.4 — Lives & Game Over	Open by playing ("Beat your score from before the break!"). Step 8 (lives), step 9 (GAME OVER sprite); step 10 (music) and tuning if time allows. Pairs tick the Project Completion checklist.
1:45	10	Mini showcase — launch day	Families rotate one table to the left. Each Director explains their rules to the visiting family, then hands off and watches. Visitors write one "liked" and one idea on a sticky note.
1:55	5	Wrap-up	Applause round. Ask the kids: "What was the hardest part, and how did you get past it?" Try at home: add one Someday List item together. Adults: journal prompts for 4.3–4.4 at home; bring them to S6.

120 Total

THE RESCUE FILE PROTOCOL (CHECKPOINT, 1:00)

The lesson's times assume a parent alone; with a child driving, expect about double. Some families *will* be behind at 1:00, and that's planned for. Say to the whole room, not to one family: "If your stars aren't being caught yet, that's fine. Here's a shortcut so you can do the fun part."

1) **File → Save to your computer** as `our-star-catcher-v1` (their work is kept). 2) **File → Load from your computer** → `14-3-star-catcher-part1.sb3`. 3) Let the child re-choose the bowl costume or backdrop right away so it feels like theirs again. 4) Rename it **Star Catcher**. The rescue file is exactly the end of step 7, so step 8 starts cleanly for everyone.

If a game is broken at 1:45, launch from the last version that worked; the showcase is about the Director explaining their game, not about which steps got finished.

WHEN A PARTICIPANT COMES WITHOUT A CHILD

Expect a few (a sick child, grown children, a grandparent whose grandkids live away). Pair two such adults as director and typist using the 4.3 kid card below, switching roles at the break. Or make them **floating playtesters** who visit tables after each checkpoint. Never leave them building alone at the back.

FAMILY BUILD NIGHT INVITATION (COPY, ADAPT, SEND AFTER S4)

You're invited: Family Build Night! On [day, date] from [time] at [place], bring your child (ages about 5–12) to the Scratch for Parents workshop. Together you'll build **Star Catcher**, a real video game where you slide a bowl to catch falling stars, and finish with a mini launch party where kids show off their games.

Your child is the **Director** (they decide what the game looks like and how it plays) and you're the **Typist** (you help with the fiddly parts, only when asked). No experience needed for kids; everything they need is on the computer we provide.

Before you come: fill in the Game Design Card with your child (it takes 10 minutes and it's half the fun). **Please bring:** the card, your Scratch sign-in if you have one, and a water bottle. Snacks provided; tell us about allergies. Siblings are welcome to share one computer. Kids don't need their own Scratch account tonight. RSVP to [contact] with your child's first name and age.

4.1 Clones — One Sprite, Many Copies

40 min • S4

◆ OBJECTIVES

- ◆ Explain a clone with the rubber-stamp idea (original = stamp, clones = prints)
- ◆ Use **create clone of myself, when I start as a clone**, and **delete this clone**
- ◆ Build the "hidden original stamps clones in a forever loop" pattern
- ◆ Know what a clone copies (position, costume, size, *visibility*) and why clones must be deleted (limit $\approx$ 300)

◆ TEACHING SEQUENCE

- 1 Why one sprite isn't enough (3m).** "In 3.2 one apple fell. What if we wanted twenty?" Duplicating 20 sprites means editing 20 times. Clones: write the code once, make copies while the program runs.
- 2 The stamp (4m).** Hold up a real rubber stamp (or cookie cutter). Press three prints. *Demo script:* "The stamp stays in my hand. The prints are on the paper. In Scratch the original sprite is the stamp and it usually hides; the clones are the prints."
- 3 The three blocks, live (5m).** Show them at the very **bottom of Control** (adults scroll past them). Build the two-script shape: original **hide** → **forever** → **create clone of myself** → **wait 1 seconds**; clone **when I start as a clone** → **show** → **say "I'm a copy!" for 2 seconds** → **delete this clone**. Delete the **wait** briefly to show the flood, then put it back.
- 4 The four rules (3m).** Clones copy everything, including being hidden (so clones need **show**); each clone runs its own copy of the clone script; **when**  **clicked** runs only on the original; always clean up (about 300 clones max; the stop sign clears them all). Ask the room: "Is this code running on the stamp, the prints, or both?"
- 5 Lab launch (from 15m).** Falling stars. Warn them up front: "When you click the flag after the first script, *nothing* happens. That's correct."

◆ LAB — FALLING STARS (17 MIN)

New project; add the **Star** sprite and a dark backdrop (Stars or Night City). Original: **hide**, then **forever go to x: (pick random -220 to 220) y: 180** → **create clone of myself** → **wait 1 seconds**. Clone: **show** → **repeat until ((y position) < -170)** → **change y by -5**, then **delete this clone**. Early finishers: random size and color, spinning stars, or snow. Reference: [labs/module-4/l4-1-falling-stars.sb3](#). **Everyone saves it:** it feeds the capstone.

Circulate for: **show** missing from the clone script (nothing appears); the falling code under the green flag (the original falls); no **go to** before stamping (stars clump in one spot); **y position** dropped in the wrong slot of the <

block; **until touching edge** used instead (each star deletes itself instantly because it's born touching the top).

◆ TEACH-IT ROLE-PLAY – "STAMPS AND PRINTS" (6 MIN)

Pairs: one adult takes the kid card, the other coaches. Coach's goal: let the child *discover* the invisible-clone bug instead of fixing it. Swap once at 3 minutes.

KID CARD — THEO, AGE 9

Excited and fast. You've just built the stamp script and clicked the flag. Nothing appears, and you say "It's broken! Scratch is broken!" If your coach grabs the mouse, fold your arms. If they ask you a question about the stamp, think out loud and find it yourself. Later, try putting **create clone of myself** inside the clone script "to get more stars."

SAY: *Where's the stamp right now? In my hand, not on the paper. So what should each print do after it lands?*

SAY: *We hid the stamp. The prints are copies of the stamp... so what do the prints look like right now?*

SAY: *Who should do the stamping — the stamp or the prints?*

Debrief (1 line each): What did it feel like not to fix it? Which question worked?

◆ WATCH-OUTS

- ◆ **"I'm lost" fog.** Clones are the first time lots happens at once, and adults who kept up easily in Module 3 may freeze. Name it: "Confusion here is your brain stretching. The stamp-and-prints question solves most of it."
- ◆ **Can't find the blocks.** The clone blocks sit below the loops and ifs at the very bottom of Control; the hat block only peeks in. Show where to scroll.
- ◆ **Trackpad drags into the wrong slot.** Dropping **y position** into the < block is fiddly. Mice help; so does zooming the code area (the + magnifier bottom-right).
- ◆ **Flood of clones.** Someone will remove the **wait** or stamp from inside the clone script. The red stop sign clears all clones; reassure them nothing is broken.
- ◆ **Screen-time guilt** surfaces when adults see how absorbing this is. Reframe: making a game is creating, not consuming.

◆ CHECK — QUIZ

Answers: clones don't appear because **they start hidden like the original and need show** (B); the script that runs only on clones is **when I start as a clone** (C); slowing then stopping = **clones never deleted, piling toward**

the 300 limit (A); create clone inside the clone script = **every clone makes another, flooding the screen until the limit (B).**

◆ OBJECTIVES

- ◆ Explain why naming a chunk of code helps (write once, change once), the recipe-card idea
- ◆ Use **My Blocks** → **Make a Block**, and put steps under the pink **define** hat
- ◆ Add a number input and drag its oval into the script
- ◆ Build the **reset game** block used in the capstone

◆ TEACHING SEQUENCE

- 1 **The recipe card (2m).** "'Make pancakes' stands for crack, whisk, pour, flip. You write the steps once, then just say the name." Programmers call it a *function*; the kids never need the word.
- 2 **Make a Block, live (4m).** Pink **My Blocks** circle → **Make a Block** → type **jump** → OK. Point at the two things that appear: the new block in the palette and the **define jump** hat. Under it: **repeat 10 → change y by 10**, **repeat 10 → change y by -10**. Call it from **when space key pressed** and **when this sprite clicked**. Then change 10 to 15 once: "Both jumps changed."
- 3 **Screen refresh (1m).** Tick **Run without screen refresh** via right-click → Edit: the cat teleports. Untick it. "Leave this unticked."
- 4 **Inputs (4m).** Make **dance** + **Add an input (number or text)** named **times** + **Add a label** "times." *Demo script:* "The pink oval in the define hat holds the number. Drag it out: you get a copy. Don't type the word." Call **dance 3 times**, then **dance 6 times**.
- 5 **Reset game preview (1m).** "This one you'll use at Family Build Night." Note that My Blocks belong to one sprite.

◆ LAB — THREE CUSTOM BLOCKS (15 MIN)

In a new cat project: (1) **jump** used by two scripts; (2) **dance (times) times** called with two different numbers; (3) make **score** and **lives** variables, then **define reset game**: set score to 0, set lives to 3, go to x: 0 y: -150, show, called from **when**  **clicked**. Challenge: **jump (height)** using **(height / 10)**. Reference: [labs/module-4/14-2-my-blocks.sb3](#).

Circulate for: steps sitting loose instead of snapped under the **define** hat (block "does nothing"); the word *times* typed into a slot instead of the oval dragged out; screen refresh ticked; a different sprite selected ("my block vanished").

◆ TEACH-IT ROLE-PLAY — "NEW WORDS FOR THE COMPUTER" (6 MIN)

Coach opens with the unplugged "zomba" game, then helps the child spot a repeat without pointing at it. Swap at 3 minutes.

KID CARD — PRIYA, AGE 11

Confident and a bit impatient. Your game has the same four blocks copied in three places. When your coach suggests a custom block, say "Copying is faster. Why bother?" Only get interested if they let you name the block (you want to call it "boing"). Later, type the word *times* into the repeat slot and insist it's right.

SAY: *I'm going to teach you a new word: zoomba. Zoomba means clap twice and spin around. Ready? Zoomba! Now let's teach the computer a new word.*

SAY: *I see those same blocks twice. What would you call what they do? What if that were one block with that name?*

SAY: *The value lives in this pink oval. Drag it out and see what happens.*

◆ WATCH - OUTS

- ◆ "Is this programming now? I'm not a programmer." Some adults hit a wall at the word *function*. Keep it to the recipe card; don't lecture on terminology.
- ◆ **Dragging the input oval** needs a precise grab on a small shape. Zoom in, bring a mouse.
- ◆ **Variables must exist first.** **reset game** needs **score** and **lives**, made **For all sprites**, as in 3.2.
- ◆ **Editing a block** (right-click the define hat → Edit) updates every use. Adults worry they "broke" the calls; they didn't.

◆ CHECK — QUIZ

Answers: the block's steps go **under the pink define block** (B); the cat teleports because **Run without screen refresh is ticked** (A); reset game missing on the Star because **custom blocks belong to the sprite they were made in** (C); to use an input, **drag the pink oval out of the define hat** (B).

◆ OBJECTIVES

- ◆ Plan a game on paper: brainstorm, pick ONE idea, sketch, list the pieces, keep a Someday List
- ◆ Fill in the Game Design Card with a child, in the child's words
- ◆ Build the **smallest playable version**: bowl moves, stars fall, catching scores
- ◆ Work as **director** (child) and **typist** (parent)

◆ TEACHING SEQUENCE

- 1 **In S4 (the 15-min briefing)**. Show the five-step planning loop and the Star Catcher design card. Hand out blank cards: "Fill it in at home with your child. Write their words, not yours." Introduce director/typist and practice the "narrate and hand it back" line in pairs for one minute.
- 2 **S5, steps 1–2 (≈ 8m)**. Delete the cat; add **Bowl** and **Star**; Stars backdrop. Variables **score** and **lives**, **For all sprites**.
- 3 **Step 3 (≈ 8m)**. On the Bowl: **define reset game** (set score to 0, set lives to 3, go to x: 0 y: -150, show). "Same block as last week."
- 4 **Step 4 (≈ 8m)**. Bowl: **when**  **clicked** → **reset game** → **forever** with two **if key (right/left arrow) pressed?** **then change x by (10 / -10)**. "Directors: click the stage, then use the arrows."
- 5 **Step 5 (≈ 6m)**. Star factory: **hide** → **wait 1 seconds** → **forever** → **create clone of myself** → **wait (pick random 0.5 to 1.5) seconds**.
- 6 **Step 6 (≈ 7m)**. Each clone: **when I start as a clone** → **go to x: (pick random -220 to 220) y: 170** → **show** → **forever** → **change y by -5**. Stars pile up at the bottom: "That's a cliffhanger for after the break."
- 7 **Step 7 (≈ 8m)**. Add the Pop sound. Inside the forever: **if touching Bowl?** **then** → **change score by 1** → **hide** → **play sound Pop until done** → **delete this clone**. Explain once: deleting a clone cuts off its sound.

◆ LAB — STAR CATCHER PART 1 (BUILT LIVE, FAMILY PAIRS)

Steps 1–7 above, paced from the projector. The finish line is the **checkpoint**: a game that catches stars and keeps score. Rescue file = end of step 7: `labs/module-4/14-3-star-catcher-part1.sb3`.

Circulate for: *who is holding the mouse* (more than any block); variables made "For this sprite only"; the arrow ifs outside the forever; the original Star visible and frozen at the top; **reset game** placed inside the forever; **start sound** instead of **play sound ... until done**.

◆ TEACH-IT — LIVE COACHING (AND THE ROLE-PLAY FOR ADULTS WITHOUT A CHILD)

Tonight the role-play is real. Coach the *adults*, not the kids: crouch beside the parent and whisper a line they can use. Adults without a child pair up with this card.

KID CARD — LILY, AGE 6

Thrilled to be the boss, wants every star purple and the bowl to be a cat. Can press the green flag and choose sprites but struggles to drag. Wants to add "a dragon that eats the stars" in the middle of step 5. After 20 minutes you get wiggly and say "you do it."

SAY: *You're the boss of this game. I'm your helper. Tell me what you want, and if you want me to do a tricky part, just say "your turn."*

SAY: *Okay, I'm finding "change x by"... it's in blue Motion... snapping it inside the if. Your turn — press the flag and see if it worked!*

SAY: *Ooh, a dragon? Someday List! We'll add it once the stars work.*

◆ WATCH - OUTS

- ◆ **Parents taking over.** The biggest risk of the night. Watch hands: if a parent has had the mouse for 3 minutes, whisper, "Can your director press the flag on this one?"
- ◆ **Parents comparing.** Families move at wildly different speeds; a six-year-old family will be slower, and that's correct. Say it aloud once: "Fast doesn't mean better tonight. The game that's *theirs* wins."
- ◆ **Big ideas mid-build.** Use the Someday List poster; a sticky note makes "not now" feel like "yes, later."
- ◆ **Scratch accounts for kids.** Don't create child accounts tonight. Save to the parent's account or **File → Save to your computer**. Account and sharing decisions come in Lesson 5.2.
- ◆ **Keyboard focus.** "The bowl won't move" is almost always "click the stage first."
- ◆ **Noise.** Ten Pop sounds at once is a lot. Keep volumes low; it's fine to add sound last.

◆ CHECK — QUIZ

Answers: huge idea → **pick ONE main action, build the small version, Someday List the rest** (B); the original Star hides because **it's a hidden factory; only clones fall and get caught** (A); hide and **play sound until done** first because **deleting a clone cuts off its sound** (B); the smallest playable version is **bowl moves + stars fall + catching adds score** (A).

◆ OBJECTIVES

- ◆ Reopen a saved project and restart momentum by playing first
- ◆ Use a lives check inside the clone, then **broadcast game over**
- ◆ Show a Game Over sprite, then **stop all**; tune the game until it feels right
- ◆ Hold a launch day where the child explains the game and watches someone play

◆ TEACHING SEQUENCE

- 1 **Play first (2m).** "Before we add anything, can you beat your score from before the break?"
- 2 **Step 8 — missing costs a life ($\approx 10m$).** Inside the clone's forever, under the catch if: **if $\{(y \text{ position}) < -170\}$ then $\rightarrow$ change lives by -1 $\rightarrow$ if $\{(lives) = 0\}$ then broadcast game over $\rightarrow$ delete this clone.** Make **game over** with **New message**. *Demo script:* "The star knows it was missed, so the star takes the life. And the delete goes *last* inside that if, or it keeps taking lives."
- 3 **Step 9 — GAME OVER ($\approx 10m$).** Paint a new sprite, Text tool, "GAME OVER", name it **Game Over**. Scripts: **when $\blacktriangleright$ clicked $\rightarrow$ hide**; **when I receive game over $\rightarrow$ go to front layer $\rightarrow$ show $\rightarrow$ stop all**. "Show first, then freeze."
- 4 **Step 10 — music & tuning ($\approx 8m$, optional).** Stage $\rightarrow$ Sounds $\rightarrow$ a *Loops* sound; **when $\blacktriangleright$ clicked $\rightarrow$ forever $\rightarrow$ play sound (loop) until done**. Tuning: -5 $\rightarrow$ -3 is easier; directors decide. Hand out the Project Completion checklist.
- 5 **Launch-day briefing (1m),** then into the mini showcase on the run-sheet.

◆ LAB — STAR CATCHER FINISHED + LAUNCH DAY

Steps 8–10 and the Project Completion checklist, then the rotating showcase: the Director explains the rules ("Use the arrows. Catch stars. Don't miss three!"), hands off, watches, and collects one "liked" and one idea. Reference: [labs/module-4/14-4-star-catcher-final.sb3](#) (your machine only; use it to show the finished game and to compare scripts when debugging).

Circulate for: the **y position** check outside the forever (stars still pile up); **delete this clone** outside the lives if (lives drop by 2–3); broadcast and receive using different messages; no green-flag **hide** on the Game Over sprite; ages 5–7: skip lives entirely, paint the sign and pick music instead.

◆ TEACH-IT — FINISH STRONG (LIVE, AND THE ROLE-PLAY CARD)

The new skill is handling frustration near the finish. Adults without a child: switch roles from 4.3 and use this card.

KID CARD — MARCUS, AGE 8

Tired after the break. You added step 8 and now every miss takes all three lives; the game ends instantly. You say "I hate this game, it's ruined," and push the mouse away. You calm down only if your coach makes the problem small. You'd rather save and stop than be lectured.

SAY: *Before we add anything — can you beat your high score from last time? Show me!*

SAY: *It worked before we added this part, so the problem is only in this part. Let's read it together, one block at a time.*

SAY: *Let's save it and fix it with fresh eyes tomorrow. That's a real programmer move.*

◆ WATCH - OUTS

- ◆ **Tears near the finish.** Energy is lowest here. Stop on a win (step 8 working is a fine place to stop) and never mid-bug.
- ◆ **The last-minute fix from the parent.** When it's broken at 1:44, parents grab the mouse "so they have something to show." Remind the room: the showcase works with whatever version last worked.
- ◆ **Lives never reach exactly 0** if **reset game** doesn't set lives to 3, or lives were made "For this sprite only" on the Bowl (the Star then can't see it).
- ◆ **Publishing pressure.** Kids will ask to put it online. "Not tonight. Next session we learn how to share safely (Lesson 5.2)."
- ◆ **Showcase shyness.** Some children won't speak to strangers. Let them point and press the flag while the parent reads the design card aloud.

◆ CHECK — QUIZ

Answers: lives drop by 2–3 because **delete this clone isn't inside the lives if** (B); the Game Over sprite shows first **so the sign is on screen when everything freezes** (A); game breaks near the finish → **shrink it to the last change, or save and stop** (B); GAME OVER still visible at the start of a new game → **add when  clicked → hide to the Game Over sprite** (C).

CLOSING THE MODULE

MODULE 4 WRAP-UP

Across two sessions the grown-ups learned clones and custom blocks, then used them with their child from a paper plan to launch day. Recap: **stamp and prints → recipe cards → design card → smallest playable version → one feature at a time → launch**. Preview S6: "Tonight you probably noticed how hard it is to keep your hands off the mouse. Next session is all about that: teaching Scratch to your child."

Before S6, note which parents struggled most with taking over; their reflections make great openers for Lesson 5.1.

Module 5 — Teaching Scratch to Your Child

How to run the final session: the coaching craft (scaffolding, questions, process praise), safe sharing in the Scratch community, ScratchJr for the under-8s, and the road ahead. Then a 25-minute family showcase and graduation. Almost no new blocks: this module teaches the *teaching*.

4 lessons

~65 min teaching + 25 min showcase

Lessons 5.1–5.4

Level Beginner

Maps to Session S6

HOW TO USE THIS GUIDE

- Same rhythm as every module: **Frame → Teach → Lab → Check**. Here the "lab" is mostly **role-play**: the kid cards and scenario cards in [labs/module-5/role-play-cards.html](#) do the heavy lifting. Print and cut them before the session.
- Talk less than any other session.** The content is short and participants already have lived experience (Family Build Night was last session). Your job is to surface that experience and let them rehearse better moves.
- Every Teach-It segment is a **pair role-play**: one adult plays the child from a kid card, the other coaches using the **say-this** lines. Swap after each round. A third person (in trios) observes with the look-fors in the answer key.
- Teal** = hands-on, **amber** = watch-outs.
- Half-day alternative:** on the 3-Saturday format, S6 is the second half of Saturday 3. Run the run-sheet below unchanged; invite the kids to arrive for the final 25 minutes.

THE THROUGH-LINE

One idea connects all four lessons: **the child drives, the grown-up coaches**. 5.1 teaches the coaching moves; 5.2 applies them to going public (the child shares, the parent sets up the guard-rails); 5.3 adapts them for a five-year-old (you read, they tap); 5.4 keeps them going after the course ends (follow their fire, keep asking "show me what you made"). The showcase is the proof: the *children* present, the parents cheer.

MODULE AT A GLANCE

LESSON	TOPIC	TIME	PARTICIPANTS LEAVE ABLE TO...
5.1	The Art of Not Taking Over	20m	Scaffold, ask instead of tell, praise the process, use "yet"

LESSON	TOPIC	TIME	PARTICIPANTS LEAVE ABLE TO...
5.2	Sharing Safely & the Scratch Community	18m	Set up a safe account and write family sharing rules
5.3	ScratchJr — Coding for Ages 5–7	17m	Build a tap-to-talk pet and coach a pre-reader
5.4	What Comes After Scratch	10m	Pick a next step on the road map with their child
S6	Showcase & Graduation	25m	Celebrate, with the children presenting their own work

SESSION S6 RUN-SHEET

CLOCK	MIN	SEGMENT	WHAT HAPPENS
0:00–0:10	10	Warm-up / check-in	Journal share from Family Build Night: "One moment you kept your hands off the mouse, and one moment you didn't." Normalize the slips; they are the raw material for 5.1.
0:10–0:30	20	5.1 The Art of Not Taking Over	Mini-teach (8) → role-play rounds with kid cards (10) → debrief (2)
0:30–0:48	18	5.2 Sharing Safely	Private-by-default demo (6) → safety checklist + family rules (7) → scenario role-play (5)
0:48–0:58	10	Break	Set out tablets for ScratchJr; set up the showcase projector and display laptop.
0:58–1:15	17	5.3 ScratchJr	Tour + differences (5) → build "My Pet Says Hi" (7) → 5-year-old role-play (5)
1:15–1:25	10	5.4 What Comes After Scratch	Road map + blocks-to-Python (5) → "next project" plan in the workbook (5)
1:25–1:35	10	Wrap-up	Final journal entry (reread the Lesson 1.1 entry first), "What Next" handout, course evaluation. Children arrive and settle.
1:35–2:00	25	Showcase & graduation	Children present (14) · kind-comment gallery · certificates (6) · welcome/close (5)
Total	120		

BEFORE THE SESSION

Print and cut **role-play-cards.html** (one set per 6 participants) and print **safety-checklist.html** (one per family). Borrow 3–6 tablets with ScratchJr pre-installed (free; iPad, Android, Chromebook). Ask families in the S5 wrap-up to bring their Star Catcher (account or **.sb3** on a USB stick) and to bring the child at 1:35. Print certificates. Have a photo-consent sign-up sheet: no photos of children without a parent's yes (it's lesson 5.2 in action).

5.1 The Art of Not Taking Over

20 min

◆ OBJECTIVES

- ◆ Use **scaffolding**: the least help that unblocks, then fade it (I lead → they lead, I prompt → they do it, I watch → they teach)
- ◆ Ask **guiding questions** instead of giving answers
- ◆ Praise **effort and strategy**, not talent; use "yet"
- ◆ Keep a child in **productive struggle**: raise the bar when bored, shrink the step when frustrated, break when boiling over

◆ TEACHING SEQUENCE

- 1 **Hook from the warm-up (2m)**. "Hands up if you grabbed the mouse at Family Build Night." Raise your own hand first. "Every time we fix it for them, we quietly teach 'this is too hard for you.'"
- 2 **The scaffold ladder (2m)**. Draw the four rungs on the board. Then the three-column contrast: "*Figure it out*" (*walks away*) · "*Which category might have a move block?*" · "*Drag the blue block here.*" Ask the room which one is the scaffold.
- 3 **Live-demo script: ask, don't tell (2m)**. Open any buggy project (the `labs/module-3/l3-3-bug-hunt-starter.sb3` file is ideal). Have a volunteer drive. You sit on your hands, visibly, and use only the five toolkit questions: "What do you want to happen? What happened instead? Which block is in charge of that? What could we try to test it? Where have we seen this before?" Count to five out loud the first time you pause.
- 4 **Process praise + "yet" (2m)**. Read the "say this / go easy on this" pairs from the slide. One line: "Praise what they *did*, because they can do it again." Model a "yet" sentence.
- 5 **Role-play launch (10m + 2m debrief)**. See Lab and Teach-It below.

◆ LAB — HANDS-OFF COACHING ROUNDS

Pairs, one laptop, one buggy project (`labs/module-3/l3-3-bug-hunt-starter.sb3` , or any participant's half-finished Star Catcher). The "child" holds the mouse; the coach may not touch it. Two 4-minute rounds, swapping roles and drawing a new kid card each time. Cards: `labs/module-5/role-play-cards.html` (kid cards K1–K6).

Circulate for: coaches whose hands drift to the mouse or trackpad; questions that are really answers ("Don't you think it should be *mouse x?*"); "you're so smart"; coaches who never shrink the step when the "child" escalates.

◆ TEACH-IT ROLE-PLAY

Kid card K1 (demo it yourself first): *Maya, 9. Frustrated. Her Star Catcher stars fall but the score never goes up. After one minute she pushes the laptop toward you: "Just do it for me."*

SAY: *What do you want to happen when a star hits the bowl? ... And what happens instead?*

SAY: *Let's read the star's script out loud together. When does it check for the bowl?*

SAY: *How about I'm your hands and you're the boss? I'll only drag the blocks you point to.*

SAY: *You tested one block at a time until you found it. That's exactly what real coders do.*

SAY: *You can't make it score... yet. Let's shrink it: just get one star to say "caught!" first.*

Debrief question: "Coaches, what was hardest? 'Children', which sentence made you want to keep going?"

◆ WATCH - OUTS

- ◆ **Guilt spiral.** Someone will say "I've said 'you're so smart' a thousand times." Answer from the lesson: warmth matters more than perfect wording; just add process praise alongside it. The shift happens over months.
- ◆ **"Isn't it mean to watch them struggle?"** Struggling *alone* would be. Sitting beside them, asking questions and shrinking the step is support that trusts them.
- ◆ **Over-acting.** Adults playing a child can go full tantrum. Keep it realistic; the card's behavior is the brief, not a stage direction.
- ◆ **Trackpad reflex.** On a laptop the coach's hand is inches from the trackpad. Have coaches literally sit on their hands or hold a cup.
- ◆ **Phones.** This is the session where family logistics buzz loudest (pick-up for the showcase). Offer a 30-second "text your family the arrival time" moment in the warm-up so phones can go face-down.

◆ CHECK - QUIZ

Answers: scaffolding = **just enough help, then fade it**; best resilience praise = **"You kept trying different things until it worked"**; bored child → **offer a new challenge** ("What if it kept score?"); child says "just do it for me" → **be their hands while they're the boss.**

◆ OBJECTIVES

- ◆ Explain accounts, the parent email, and why Scratch collects minimal data (non-profit, COPPA-compliant)
- ◆ Show that projects are **private until the child clicks Share**, and that an account is optional
- ◆ Name the community protections: moderation, guidelines, Report, comments off, delete, no private messaging
- ◆ Draft family sharing rules and hold the "before sharing" conversation

◆ TEACHING SEQUENCE

- 1 Name the worry (1m).** "This is the part parents worry about most. Let's look at exactly how it works." Worry is a sign of paying attention, not paranoia.
- 2 Live-demo script: private by default (5m).** Logged in on the projector, open My Stuff. Point to an unshared project: "Nobody can see this." Open it, point at the orange **Share** button: "Nothing goes public until your child clicks this." Then open a shared project page and show the comment area, the **Report** button, and the "Comments on/off" toggle. Say plainly: "There are no private messages on Scratch. Everything is public and visible to you." Finish on scratch.mit.edu/parents as the official source.
- 3 Safety checklist + family rules (7m).** Hand out [labs/module-5/safety-checklist.html](https://scratch.mit.edu/labs/module-5/safety-checklist.html). Participants tick what they've already done and draft three or four family rules in the box: what never gets shared, where Scratch is used, what to do with a mean comment.
- 4 Scenario role-play (5m).** See Teach-It below.

◆ LAB – SAFETY CHECKLIST & FAMILY SHARING RULES

Each family completes [labs/module-5/safety-checklist.html](https://scratch.mit.edu/labs/module-5/safety-checklist.html) and drafts their sharing rules to sign with their child at home. Then pairs draw one scenario card (S1–S8) from [labs/module-5/role-play-cards.html](https://scratch.mit.edu/labs/module-5/role-play-cards.html).

Circulate for: usernames that include a real name or birth year; rules phrased only as "don'ts" (nudge toward "if anyone's unkind, show me and we'll handle it together"); parents who assume sharing is required (it isn't).

◆ TEACH-IT ROLE-PLAY

Kid card K7: *Leo, 11. Excited. Wants to share Star Catcher right now so his friends can play it. He's picked the username "LeoMartinez2014".*

SAY: *I love that you want people to play it. Before we share, let's pick a username that doesn't say who*

you are. Something about stars or cats?

SAY: *When we share, kids around the world can see it and comment. Most are super nice! Our family rules: no real name, where we live, or our school, and if anyone's ever unkind, you show me and we handle it together. Deal?*

SAY: *Your game started from the course, so let's add a thank-you in the Notes and Credits. Crediting others is what good Scratchers do.*

◆ WATCH - OUTS

- ◆ **Home tech-support questions** ("How do I lock down my son's Chromebook?"). Park them on a "Parking lot" sheet; answer after class or point to the device maker. Stay on Scratch.
- ◆ **Screen-time guilt.** Reframe: Scratch time is *making* time. The healthy-habits list says nudge "let's make" over "let's watch" — that's the lever, not a total ban.
- ◆ **Stale specifics.** Scratch's policies and features change. Don't improvise details you're unsure of: send people to the official Parents, Community Guidelines and Privacy Policy pages.
- ◆ **Over-promising.** Moderation is strong but not perfect. Say so; the family rule for mean comments is the real safety net.

◆ CHECK - QUIZ

Answers: public **only after they click Share**; the community is **moderated, has guidelines, and has no private DMs**; best young-sharer habit = **non-identifying username, your email on the account, regular check-ins**; unkind comment → **read it together, report it, delete it, talk about how it felt.**

◆ OBJECTIVES

- ◆ Explain how ScratchJr differs from Scratch: picture blocks in a **left-to-right row**, grid squares, tablets, no online community
- ◆ Recognize the six colors: yellow Triggering, blue Motion, purple Looks, green Sound, orange Control, red End
- ◆ Build the tap-to-talk pet ("My Pet Says Hi")
- ◆ Coach a 4–7-year-old: 10–15-minute sessions, story first, you read / they tap

◆ TEACHING SEQUENCE

- 1 **Differences in two minutes (2m).** Same big ideas (events start scripts, blocks run in order, loops repeat); different everything else. Point out the one that relaxes parents: **projects stay on the device**, no comments or strangers.
- 2 **The color trap (1m).** In Scratch green = Operators and sound is pink. In ScratchJr green = Sound and yellow starts things. "Trust the pictures."
- 3 **Live-demo script (2m).** Mirror a tablet if you can (or hold it under a document camera). Tap +, pick an animal. Yellow → drag **Start on Tap**. Blue → snap **Hop** to the right. Purple → **Grow**. Green → microphone, record "Hi! I'm your pet!". Purple → **Shrink**. Red → **End**. Tap the pet. Then say: "Without Shrink, it gets bigger with every tap."
- 4 **Build it (7m).** In twos or threes per tablet. Fast finishers add page 2 with **Go to Page**.
- 5 **5-year-old role-play (5m).** See Teach-It below.

◆ LAB — "MY PET SAYS HI"

Build the row *Start on Tap* → *Hop 2* → *Grow 2* → *My Sound* → *Shrink 2* → *End* on a tablet. No Scratch file: ScratchJr projects live on the device. **No tablets?** Use the cut-out paper ScratchJr blocks at the end of labs/module-5/role-play-cards.html: participants lay out the row on the table left to right and "run" it by acting it out.

Circulate for: blocks not snapped into one row (nothing happens on tap); Green Flag used instead of Start on Tap; Grow with no Shrink; silent recordings (microphone permission off in the tablet's settings).

◆ TEACH-IT ROLE-PLAY

Kid card K8: *Ava, 5. Can't read yet. Wiggly. Wants to record a burp for the puppy and keeps dragging blocks back onto the palette (which deletes them).* The coach's hands stay in their lap; they may steady the tablet.

SAY: *Tell me a tiny story first. Who's in it, and what happens?*

SAY: *So the puppy jumps, gets really big, and barks. Which picture looks like jumping? ... Yes, the hop! What comes next?*

SAY: *Whoa, that's not what we planned! What should we try?*

SAY: *You made the puppy do that! You told it what to do, and it listened. That's what coders do.*

◆ WATCH-OUTS

- ◆ **"We don't have a tablet."** ScratchJr is for tablets and Chromebooks. On a regular computer, use full Scratch with the Ages 5–7 tips: you read, they choose and press the green flag.
- ◆ **Adult fingers on a tablet.** Drag-and-drop on glass is as fiddly for some adults as the trackpad was in S1. Offer a stylus if you have one; reassure that a block dragged back to the palette is simply deleted, no harm done.
- ◆ **Color confusion** for the facilitator too: rehearse the six colors before class.
- ◆ **"Is 4 too young?"** Not with a lap to sit on and very short sessions. If it isn't fun yet, try again in a few months.

◆ CHECK – QUIZ

Answers: ScratchJr blocks sit in a **horizontal row, read left to right**; react to a touch = **Start on Tap**; session length for a 5-year-old = **about 10–15 minutes, ending while it's still fun**; pet grows with every tap → **add Shrink (same number) or Reset Size**.

◆ OBJECTIVES

- ◆ Place ScratchJr, Scratch, physical computing and text languages on one road map
- ◆ Name what's still ahead inside Scratch: **lists** (Variables → Make a List) and **extensions** (Pen, Music, Text to Speech, Video Sensing, micro:bit)
- ◆ Explain why the jump to Python is mostly **syntax**: the concepts transfer
- ◆ Leave with trusted next resources and one "next project" per family

◆ TEACHING SEQUENCE

- 1 The road map (2m).** ScratchJr (5–7) → Scratch (8+) → three branches: deeper Scratch, physical computing (Makey Makey 7+, LEGO robotics 8+, micro:bit 9+), text languages (Python, JavaScript). "There's no single correct path."
- 2 Live-demo script: the bridge (3m).** Put the Scratch stack *repeat 3 → say Hi!* beside `for i in range(3): print("Hi!")`. Ask: "What's the same?" Let the room answer: the loop, the count, the message. "Your child already owns the concepts. Python just types them." Then click **Add Extension** (bottom-left of the editor) to show Pen and Music exist.
- 3 Next-project plan (5m).** In the workbook, each participant names the next project their child is excited about and which road-map branch it leads to. Two or three share aloud.

◆ LAB — MY FAMILY'S NEXT PROJECT

Workbook page "What Next" (take-home handout). No Scratch file. Fast finishers draw a scenario card (S7–S8 fit 5.4) and rehearse with a partner.

Circulate for: plans that are the *parent's* idea ("She should learn Python") rather than the child's; nudge back to "What does your child light up about?"

◆ TEACH-IT ROLE-PLAY

Kid card K10: *Sam, 12. Bored and a bit dismissive: "Scratch is for babies. I want to do real coding."*

SAY: *You already know real coding. Look: this Scratch loop and this Python loop are the same idea, just typed.*

SAY: *What would you want to make if you could make anything? Let's find the tool that fits it.*

SAY: *Look how far you've come. Remember when we made the cat just say hello? What do you want to make next?*

◆ WATCH - OUTS

- ◆ **Parent anxiety about "falling behind."** Many kids happily stay in Scratch into their early teens. Switch when curious, not on a schedule (typing fluency and reading comfort help; many start around 10–13).
- ◆ **"My child is already better than me."** Your job changes, it doesn't end: audience, question-asker, cheerleader.
- ◆ **Buying hardware on impulse.** Suggest trying the free options first (offline app, Pen/Music extensions, Code.org, the library coding club).

◆ CHECK - QUIZ

Answers: natural next step inside Scratch = **lists, or an extension like Pen or Music**; Scratch → Python isn't scary because **the core concepts transfer; only the syntax changes**; best momentum = **follow their interests with another exciting project**; extensions live behind the **Add Extension button, bottom-left of the editor**.

◆ WRAP-UP BEFORE THE CHILDREN ARRIVE (10M)

- 1 **Final journal (6m).** "Flip back to your first entry from Lesson 1.1. Read your why, your worry and your confidence number. Now write today's number." Offer, never require, one share.
- 2 **Hand out "What Next" (2m).** The last page of the workbook: road map, the trusted resources from 5.4, the family's next project.
- 3 **Evaluation (2m).** Hand out your course evaluation while children arrive.

◆ SHOWCASE & GRADUATION (25M)

- 1 **Welcome the children (3m).** "Tonight the coders present and the grown-ups cheer." Remind families: photos only with a yes on the consent sheet.
- 2 **Showcase (14m).** Up to ~10 families: 60–90 seconds each on the projector. The **child** presents their Star Catcher (or any project) and shows one thing they're proud of; the parent adds one sentence: "A moment I kept my hands off the mouse was...". More than 10 families: run a **gallery walk** instead, laptops open around the room, with sticky notes for *kind, curious, specific* comments (lesson 5.2's gold standard: "I love how... — how did you do that?").
- 3 **Graduation (6m).** Read each name, hand over the certificate. Nice touch: the child hands the certificate to their parent.
- 4 **Close (2m).** One line: "You started this course unsure whether you were 'techy enough.' You finished it having built a game and shown your child that grown-ups learn hard things too." Point to the library or school coding club if one exists.

CERTIFICATE TEXT (SUGGESTED)

Certificate of Completion — Scratch for Parents: Learn It, Then Teach It. This certifies that _____ completed six workshop sessions (17 lessons): writing their first programs, building a complete game with clones and custom blocks, and learning the craft of coaching a young coder. Awarded on _____ · Facilitator _____.

Optional child certificate — "Official Scratch Coder": _____ designed and built a game, fixed real bugs, and taught a grown-up something new. Keep making wonderful things!

◆ WATCH-OUTS

- ◆ **A project that won't load** (no Wi-Fi, forgotten password). Have the display laptop signed out and ready for `.sb3` files from USB; failing that, the child describes it and demos next time. Never let a family end on a failed load: celebrate the attempt.

- ◆ **Shy presenters.** Offer "you drive, your parent talks" or a presenting pair of siblings. Nobody is made to speak.
- ◆ **Comparisons.** Some projects will be far more elaborate. Praise process in every introduction ("...who kept testing until the stars fell").

CLOSING THE MODULE

COURSE CLOSE

Recap the whole arc in one breath: **learn it (Modules 1–4) → build it together (Family Build Night) → teach it (Module 5)**. Remind participants that every lesson stays online to revisit, especially the editor tour, the debugging table and the safety checklist. Thank them; many were nervous in S1, and it's worth saying out loud how far they've come.